บทที่ 9 การจัดการธุรกรรมและความมั่นคงปลอดภัยของข้อมูล

วัตถุประสงค์

- สามารถอธิบายความหมายและความสำคัญของธุรกรรม/รายการเปลี่ยนแปลง (transaction) ได้
- สามารถอธิบายความหมายของภาวะพร้อมกัน (concurrency)
- สามารถอธิบายปัญหาที่เกิดจากภาวะพร้อมกันของธุรกรรม
- สามารถอธิบายวิธีแก้ปัญหาที่เกิดจากภาวะพร้อมกันของธุรกรรมได้
- สามารถอธิบายกลไกและวิธีการกู้ข้อมูลได้
- สามารถอธิบาย วางแผน และดำเนินการรักษาความมั่งคงปลอดภัยของฐานข้อมูลได้

คำสำคัญ: ธุรกรรม/รายการเปลี่ยนแปลง (transaction); ภาวะพร้อมกัน (concurrency); serializibility; เทคนิคการปิดกั้น (lock technique); การปิดตาย (dead lock); การกู้ข้อมูล (data recovery); การลงบันทึกระบบ (system log); ความมั่งคงปลอดภัยของฐานข้อมูล (database security); ภัยคุกคาม (thread)

บทที่ 9 การจัดการธุรกรรมและความมั่นคงปลอดภัยของข้อมูล

9.1 บทนำ

หลังจากที่ได้ทำการออกแบบ และพัฒนาระบบฐานข้อมูล ตลอดจนนำระบบฐานข้อมูลมาใช้แล้ว การดูแลและจัดการฐานข้อมูลเป็นสิ่งที่จะต้องปฏิบัติต่อไป ซึ่งสิ่งที่ควรคำนึงถึงในการจัดการระบบฐานข้อมูล ได้แก่ความน่าเชื่อถือของข้อมูล และความมั่นคงปลอดภัยของข้อมูล โดยเฉพาะในกรณีที่มีใช้งานฐานข้อมูลพร้อมกันหลายๆ คนหรือในกรณีที่ระบบทำงานไม่ปกติหรือเกิดความล้มเหลวของระบบ ซึ่งในบทนี้จะกล่าวถึงการจัดการภาวะพร้อมกัน (concurrency control) การกู้ข้อมูล (data recovery) และความมั่นคงปลอดภัยของข้อมูล (data security)

การจะจัดการภาวะพร้อมกันจะต้องเข้าใจถึงลักษณะการใช้งานที่ก่อให้เกิดปัญหาขึ้น โดยจะต้องเข้าใจถึงธุรกรรม (transaction) และ serializibility จากนั้นจะสามารถเข้าใจถึงการใช้เทคนิคต่างๆ ในการแก้ปัญหาภาวะพร้อมกันได้ โดยเฉพาะการใช้เทคนิคการปิดกั้น (lock technique) จะการจัดการกับปัญหาที่อาจจะตามมาอีกในการใช้เทคนิคต่างๆ ซึ่งได้อธิบายในหัวข้อ 9.2 สำหรับหัวข้อ 9.3 เป็นการอธิบายกลไกของการกู้ข้อมูลของระบบจัดการฐานข้อมูลในกรณีที่ระบบทำงานไม่ปกติ และหัวข้อ 9.4 อธิบายถึงการรักษาความมั่นคงปลอดภัยของข้อมูลด้วยวิธีต่างๆ เพื่อให้ข้อมูลปลอดภัยจากภัยคุกคามต่างๆ

9.2 การจัดการภาวะพร้อมกัน

การรองรับการทำงานของผู้ใช้งานพร้อมๆ กันหลายคนอย่างมีประสิทธิภาพเป็นข้อดีที่สำคัญข้อหนึ่งของระบบจัดการฐานข้อมูล อย่างไรก็ตามการใช้งานฐานข้อมูลพร้อมๆ กันโดยผู้ใช้งานหลายคนก่อให้เกิดปัญหาที่จะต้องจัดการ โดยเฉพาะการใช้งานข้อมูลบางข้อมูลร่วมกันและทำให้ข้อมูลที่ผู้ใช้อีกคนหนึ่งใช้อยู่นั้นเป็นข้อมูลที่เป็นปัจจุบัน ในที่นี้จะเป็นการอธิบายถึงกลไกอันเป็นพื้นฐานของการจัดการภาวะพร้อมกัน ซึ่งระบบจัดการฐานข้อมูลอาศัยกลไกนี้ในการจัดการกับภาวะพร้อมกันแล้วระดับหนึ่ง ซึ่งผู้พัฒนาระบบสารสนเทศที่ใช้งานฐานข้อมูลต้องพัฒนาระบบให้สอดรับกับกลไกในการจัดการภาวะพร้อมกันนี้ ในหัวข้อนี้จะกล่าวถึงแนวคิดที่เกี่ยวข้องกับการจัดการภาวะพร้อมกันได้แก่ ธุรกรรม (transaction) ความจำเป็นในกาแก้ปัญหาภาวะพร้อมกัน serializibility และเทคนิคทั่วไปที่ใช้ในการจัดการกับภาวะพร้อมกัน

9.2.1 ธุรกรรม/รายการเปลี่ยนแปลง (Transaction)

ธุรกรรม หรือ รายการ(เปลี่ยนแปลง) ในที่นี้คือธุรกรรมฐานข้อมูล (database transaction) หมายถึงคำสั่ง หรือชุดของคำสั่งที่กระทำกับฐานข้อมูลเพื่อให้บรรลุกิจกรรมใดๆ ตามวัตถุประสงค์ โดยกำหนดว่าคำสั่งย่อยๆ ต้องดำเนินการให้เสร็จสิ้นเพื่อธุรกรรม 1 ธุรกรรม และการดำเนินธุรกรรมของผู้ใช้แต่ละคนพร้อมกันนั้นเป็นอิสระต่อกัน ตลอดจนเสมือนกับผู้ใช้แต่ละคนใช้งานฐานข้อมูลอยู่เพียงคนเดียว การจัดการธุรกรรมในฐานข้อมูลนั้นมีวัตถุประสงค์ 2 ข้อใหญ่ๆ ได้แก่

1) เพื่อสร้างกลไกในการกู้คืนข้อมูล ในกรณีที่ธุรกรรมกำลังดำเนินการยังไม่เสร็จสิ้นไม่ว่าจะด้วยเหตุการณ์ใดๆ ข้อมูลบางส่วนได้ถูกเปลี่ยนแปลงแต่ยังไม่ครบถ้วน จะต้องจัดการปรับสภาพ

ข้อมูลในฐานข้อมูลให้ถูกต้อง ไม่ว่าจะเป็นการยกเลิกการแก้ไขที่ได้กระทำมา หรือดำเนินต่อให้เสร็จสิ้น ฯลฯ

2) เพื่อสร้างความเป็นอิสระแยกจากกันของผู้ใช้งานฐานข้อมูลหลายๆ คนพร้อมกัน ไม่มีการแก้ไขข้อมูลซ้ำกันจนเป็นเหตุให้เกิดความผิดพลาด

ซึ่งวัตถุประสงค์ทั้ง 2 ข้อนี้ของการจัดการธุรกรรมคือใจความสำคัญของบทนี้ ธุรกรรมของฐานข้อมูลจะต้องมีคุณสมบัติที่เรียกสั้นๆ ว่า ACID กล่าวคือการไม่แบ่งแยก (atomicity) ความสอดคล้องต้องกันของข้อมูล/ความต้องกัน (consistency) ความเป็นอิสระแยกจากกัน (isolation) และการคงอยู่อย่างถาวร/ความคงทน (durability) (WikiPedia, 2008)

ธุรกรรมมีลักษณะ “ทั้งหมดหรือไม่ทำเลย (all-or-nothing)” หมายความว่าธุรกรรมต้องทำสำเร็จทั้งหมด ไม่เช่นนั้นก็เสมือนกับว่าไม่มีการดำเนินคำสั่งย่อยใดๆ เลย ข้อมูลที่กระทำโดยคำสั่งย่อยๆ ทั้งหมดต้องเป็นไปตามข้อกำหนดบูรณภาพเพื่อความถูกต้องของฐานข้อมูลและข้อมูลต้องสอดคล้องต้องกันในสภาพที่ควรจะเป็นในบริบทการใช้งาน ธุรกรรมแต่ละธุรกรรมต้องเป็นอิสระแยกจากกันและข้อมูลที่เปลี่ยนแปลงจากการดำเนินการที่เสร็จสิ้นของธุรกรรมต้องคงอยู่ในอุปกรณ์จัดเก็บต่อไป

คำสั่งย่อยๆ ในธุรกรรมได้แก่คำสั่งที่ดำเนินการกับฐานข้อมูล เช่น คำสั่งในการแสดงข้อมูล และคำสั่งในการแก้ไขข้อมูล สำหรับคำสั่งในการแสดงข้อมูลจากฐานข้อมูลนั้นได้แก่คำสั่ง SELECT ส่วนคำสั่งที่ใช้ในการแก้ไข้ข้อมูลนั้นได้แก่คำสั่ง INSERT ที่ใช้ในการเพิ่มข้อมูล คำสั่ง UPDATE ที่ใช้ในการปรับปรุงแก้ไขข้อมูล และคำสั่ง DELETE ในการลบระเบียนข้อมูล

พิจารณาตัวอย่างธุรกรรมการโอนเงิน นายสมชายจากบัญชีหมายเลข 0352000001 โอนเงินจำนวน 1,000 บาทไปให้กับนางสมศรี บัญชีหมายเลข 0352000002 ในที่นี้ balance คือชื่อฟิลด์ของข้อมูลยอดเงินในบัญชี และ bank_account คือตารางข้อมูลบัญชี ดังนี้

```
BEGIN TRANSACTION

EXEC SQL UPDATE bank_account
       SET balance = balance - 1000
       WHERE account_no = '0352000001';

EXEC SQL UPDATE bank_account
       SET balance = balance + 1000
       WHERE account_no = '0352000002';
COMMIT;
```

จากตัวอย่าง จะเห็นได้ว่าธุรกรรมการโอนเงินนี้ประกอบด้วยคำสั่ง SQL ที่ใช้ในการแก้ไขข้อมูลหลักๆ 2 คำสั่ง ได้แก่การ UPDATE ข้อมูลของบัญชี 2 บัญชี ซึ่งธุรกรรมนี้จะสำเร็จลงได้ยอดเงินในบัญชีแรกจะต้องถูกหักออก และยอดเงินในบัญชีที่ 2 จะต้องเพิ่มขึ้นให้สำเร็จเรียบร้อยจึงจะถูว่าธุรกรรมสมบูรณ์ ในกรณีที่คำสั่งใดๆ ไม่สำเร็จ ข้อมูลทั้งหมดจะต้องอยู่ในสภาพก่อนเริ่มธุรกรรม คำสั่ง BEGIN TRANSACTION เป็นคำสั่งกำหนดจุดเริ่มต้นของธุรกรรม และคำสั่ง COMMIT เป็นคำสั่งในการยอมรับธุรกรรมและทำให้ผลของการเปลี่ยนแปลงข้อมูลคงอยู่

นอกจากนี้แล้วชุดคำสั่ง SQL แต่ละคำสั่งยังประกอบไปด้วยกระบวนการย่อยๆ ที่เกี่ยวข้องกับอุปกรณ์ฮาร์ดแวร์และกลไกของระบบจัดการฐานข้อมูล ก่อนที่การดำเนินการตามคำสั่งนั้นๆ จะสำเร็จลุล่วงซึ่งสามารถอธิบายได้ดังนี้

คำสั่งที่เกี่ยวข้องกับการอ่านข้อมูลนั้นมีขั้นตอนดังนี้

<u>การอ่านข้อมูล</u> กำหนดให้การอ่านข้อมูล X (ข้อมูลสมมติ) เขียนแทนด้วย READ(X)

1. ค้นหาตำแหน่งที่อยู่ของ block ข้อมูลที่มีข้อมูล X ซึ่งเป็นตำแหน่งบนจานบันทึกข้อมูล เพื่อหาข้อมูล X
2. คัดลอกข้อมูล block ดังกล่าวมาเก็บไว้ใน buffer ในหน่วยความจำหลักก่อนนำไปประมวลผล
3. คัดลอกข้อมูล X จาก buffer ซึ่งเป็นที่พักข้อมูลไปประมวลผลยังชุดคำสั่งที่ใช้งานข้อมูล X

<u>การเขียนข้อมูล</u> กำหนดให้การเขียนข้อมูล X (ข้อมูลสมมติ) เขียนแทนด้วย WRITE(X)

1. ค้นหาตำแหน่งที่อยู่ของ block ข้อมูลที่มีข้อมูล X ซึ่งเป็นตำแหน่งบนจานบันทึกข้อมูล เพื่อหาข้อมูล X
2. คัดลอกข้อมูล block ดังกล่าวมาเก็บไว้ใน buffer ในหน่วยความจำหลักเพื่อรอการแก้ไขข้อมูล
3. คัดลอกข้อมูล X จากผลลัพธ์ที่ได้จากการประมวลผลด้วยชุดคำสั่งไปเขียนลงยัง buffer ซึ่งเป็นที่พักข้อมูล
4. เขียนข้อมูล block ที่ได้ทำการปรับปรุงนี้จาก buffer ลงยังจานบันทึกข้อมูล

ขั้นตอนที่ 4 ซึ่งเป็นขั้นตอนสุดท้ายของการเขียนข้อมูลนั้นถือว่าเป็นการดำเนินการเสร็จสิ้นของธุรกรรม เนื่องจากข้อมูลที่แก้ไขจะคงอยู่ ดังนั้นเมื่อขั้นตอนนี้สำเร็จ และไม่มีคำสั่งอื่นๆ อีกจึงจะถึงว่าธุรกรรมนั้นๆ เสร็จสมบูรณ์ ระบบจัดการฐานข้อมูลจะดำเนินการยอมรับ (COMMIT) ว่าธุรกรรมนั้นๆ สำเร็จลงแล้ว แต่หากการดำเนินธุรกรรมไม่มีเหตุขัดข้อง (failure) ก่อนการ COMMIT การดำเนินการย่อยๆ ที่ดำเนินการไปแล้วทั้งหมดจะถูก ROLLBACK หรือย้อนกลับเพื่อให้ข้อมูลมีสภาพดังเช่นก่อนเริ่มธุรกรรม

การจัดการธุรกรรมมีความจำเป็นเนื่องจากข้อมูลที่ถูกใช้งานจากธุรกรรมหลายๆ ธุรกรรมพร้อมกัน ตลอดจนความขัดข้องของระบบจะทำให้ข้อมูลไม่มีความต้องกัน (inconsistency) ได้ ดังที่จะได้อธิบายต่อไปนี้

9.2.2 ความจำเป็นในการแก้ปัญหาภาวะพร้อมกัน

การใช้งานระบบฐานข้อมูลโดยผู้ใช้หลายๆ คนพร้อมกัน อาจก่อให้เกิดปัญหาขึ้นได้ พิจารณาตัวอย่างธุรกรรมและกรณีตัวอย่างที่ก่อให้เกิดปัญหาจากภาวะพร้อมกัน ดังนี้

นายสมชายและนางสมศรีเป็นเจ้าของบัญชีธนาคารร่วมกัน มีเงินฝากในบัญชี 10,000 บาท นายสมชายและนางสมศรีทำธุรกรรมพร้อมๆ กัน โดยนายสมชายทำการฝากเงินเข้าบัญชีเป็นจำนวน 2,000 บาท นางสมศรีถอนเงินออกจากบัญชี 1,000 บาท แสดงเป็นขั้นตอนการทำงานย่อยๆ ของธุรกรรมทั้งสองด้วยโค้ดจำลอง (pseudo code) ได้ดังนี้

ธุรกรรมที่ 1 (T1) นายสมชายฝากเงิน 2,000 บาท สมมติให้ยอดเงินในบัญชีคือข้อมูล X

```
T1:     READ(X)
        UPDATE X = X + 2000
        WRITE(X)
```

ธุรกรรมที่ 2 (T2) นางสมศรีถอนเงิน 1,000 บาท

```
T2:     READ(X)
        UPDATE X = X - 1000
        WRITE(X)
```

ระบบจัดการฐานข้อมูลรองรับการทำงานในกรณีที่ธุรกรรมทั้ง 2 ทำงานพร้อมกัน หลักการทำงานของการรองรับธุรกรรมหลายๆ ธุรกรรมพร้อมกันนั้นคล้ายคลึงกันกับหลักการทำงานหลายงาน (multi-tasking) ของระบบปฏิบัติการ เช่นระบบปฏิบัติการ Microsoft Windows ที่ผู้ใช้สามารถพิมพ์เอกสารด้วย Microsoft Word พร้อมๆ กับการฟังเพลงจากโปรแกรม Media Player และรอเว็บเพจที่กำลังโหลดอยู่ด้วยโปรแกรม Internet Explorer หลักการทำงานของระบบปฏิบัติการที่ทำงานหลายๆ อย่างบนเครื่องคอมพิวเตอร์เครื่องเดียวพร้อมๆ กันนั้นอาศัยการสลับการทำงานไปมาของหน่วยประมวลผลให้ประมวลผลโปรแกรมทีละโปรแกรมด้วยช่วงเวลาสั้นๆ ซึ่งกระบวนการสลับงานเพื่อประมวลผลโปรแกรมแต่ละโปรแกรมเกิดขึ้นอย่างรวดเร็วมากและวนไปเรื่อยๆ บนเครื่องคอมพิวเตอร์ ในขณะที่หน่วยประมวลผลทำการประมวลผลโปรแกรมๆ หนึ่งอยู่และหยุดพักการประมวลผล เพื่อไปประมวลผลอีกโปรแกรมหนึ่งเราเรียกว่าการ interleave แล้ววนกลับมาประมวลผลโปรแกรมที่ค้างไว้ หลักการนี้เองเป็นหลักการเดียวกันกับการประมวลผลธุรกรรมหลายๆ ธุรกรรมพร้อมๆ กันของระบบจัดการฐานข้อมูล โดยระบบจัดการฐานข้อมูลจะทำการ interleave เพื่อประมวลผลหลายๆ ธุรกรรมอย่างมีประสิทธิภาพ โดยเฉพาะเมื่อระบบกำลังรอการจัดการข้อมูลกับอุปกรณ์จัดเก็บที่ใช้เวลานาน ผู้ใช้แต่ละคนไม่จำเป็นต้องรอการทำงานเป็นลำดับตั้งแต่ลำดับแรกจนถึงคนสุดท้าย ซึ่งใช้เวลาเกินความจำเป็นในการรอการติดต่อกับอุปกรณ์จัดเก็บ ตัวอย่างนี้เป็นข้อดีตัวอย่างหนึ่งของการให้บริการธุรกรรมพร้อมๆ กันของฐานข้อมูล

พิจารณากรณีตัวอย่างการจัดการธุรกรรมฝากและถอนเงิน T1 และ T2 ของนายสมชายและนางสมศรีด้วยหลักการการให้บริการธุรกรรมพร้อมๆ กัน ดังนี้

สมมติให้ธุรกรรม T1 และ T2 เกิดขึ้นพร้อมๆ กันและการประมวลผลรายการทั้ง 2 เป็นแบบสลับกันและเกิดการ interleave ผลลัพธ์ที่ได้จากกรณีตัวอย่างนี้ข้อมูลยอดเงินไม่ถูกต้อง ซึ่งเกิดจากเหตุผลหลักคือการอ่านค่าของธุรกรรม T2 ก่อนที่ข้อมูลที่ได้รับการแก้ไขในธุรกรรม T1 จะบันทึกลงในฐานข้อมูล

กรณีตัวอย่างเกิดขึ้นโดยธุรกรรม T1 ของนายสมชายทำการฝากเงินเริ่มอ่านข้อมูลยอดเงิน READ (X) จากฐานข้อมูล มีค่าเท่ากับ 10,000 จากนั้นระบบทำการคำนวณเพิ่มยอดเงินในบัญชี UPDATE X = X + 2,000 ทำให้ยอดเงินในขณะนี้มีค่าเท่ากับ 12,000 แต่เป็นยอดเงินที่อยู่ในหน่วยความจำของเครื่องคอมพิวเตอร์ ยังไม่ได้ปรับปรุงลงในจานบันทึกข้อมูลและยังไม่ได้ COMMIT ธุรกรรม แต่ระหว่างที่ระบบกำลังคำนวณยอดเงินใหม่นั้น ระบบจัดการฐานข้อมูลทำการ interleave เพื่อประมวลผลธุรกรรม T2 ที่เข้ามาพร้อมๆ กัน ณ เวลา t2 เพื่ออ่านข้อมูลยอดเงิน READ (X) ได้ค่ายอดเงิน 10,000 บาทซึ่งเป็นยอดเงินจากฐานข้อมูล ถึงขณะนี้ไม่ว่าธุรกรรมใดจะสำเร็จก่อนหรือหลังจะเกิดความผิดพลาดของข้อมูลขึ้น ในกรณีนี้ T2 ได้รับการประมวลผลเสร็จสิ้น โดยยอดเงินในบัญชีถูกหักออก 1,000 บาท UPDATE X = X – 1,000 เหลือ 9,000 และถูกเขียนลงในฐานข้อมูล WRITE (X) และ COMMIT จากนั้นที่เวลา t3 ระบบทำการประมวลผล T1 ที่เหลือต่อ โดยทำการเขียนข้อมูลยอดเงิน 12,000 ที่คำนวณค้างไว้ลงในฐานข้อมูลและ COMMIT ทำให้มียอดเงินเท่ากับ 12,000

ตารางที่ 9-1 ตารางกรณีตัวอย่างการดำเนินธุรกรรมที่ก่อปัญหาจากภาวะพร้อมกัน

เวลา	ธุรกรรม T1	ธุรกรรม T2	ยอดเงิน (X)
t1	BEGIN TRANSACTION		10000
	READ (X) (10000)		10000
⇨	UPDATE X = X + 2000 (**12000**)		**10000**
t2		BEGIN TRANSACTION	10000
		⇨ READ (X) (10000)	10000
		UPDATE X = X – 1000 (9000)	10000
		⇨ WRITE (X) (9000)	9000
		COMMIT	**9000**
t3 ⇨	WRITE (X) (12000)		**12000**
	COMMIT		12000

หากลองคำนวณอย่างง่ายจะพบว่าการฝากเงิน 2,000 และถอนเงิน 1,000 บาท ยอดเงินในบัญชีควรจะมีค่าเท่ากับ 11,000 บาท แต่ในกรณีนี้มีเงิน 12,000 บาทในบัญชี เกินมา 1,000 บาท ซึ่งอาจจะเป็นที่ชื่นชอบของนายสมชายและสมศรี แต่ธนาคารจะเกิดความเสียหาย ในทางกลับกันถ้าเกิดธุรกรรมไม่ได้เกิดขึ้นตามลำดับในกรณีตัวอย่างนี้แต่เป็นกรณีที่ T2 เป็นฝ่ายเขียนข้อมูลลงฐานข้อมูลทีหลัง เงินในบัญชีจะเหลือเพียง 9,000 บาทเท่านั้น

ตัวอย่างนี้เองเป็นการอธิบายถึงความสำคัญของการจัดการภาวะพร้อมกันเพื่อไม่ให้ข้อมูลผิดพลาด ซึ่งปัญหาที่เกิดจากภาวะพร้อมกันที่ไม่ได้จัดการอย่างเหมาะสมแบ่งเป็น 3 ลักษณะดังนี้

1) ปัญหาข้อมูลที่ถูกปรับปรุงถูกเขียนทับ

ปัญหานี้มีลักษณะดังตัวอย่าง T1 และ T2 การฝากและถอนเงินของสมชายและสมศรี

2) ปัญหาการอ้างอิงข้อมูลที่ไม่ได้ COMMIT

ยกตัวอย่าง

กรณีตัวอย่างเกิดขึ้นโดยธุรกรรม T1 ของนายสมชายทำการฝากเงินเริ่มอ่านข้อมูลยอดเงิน READ (X) จากฐานข้อมูล มีค่าเท่ากับ 10,000 จากนั้นระบบทำการคำนวณเพิ่มยอดเงินในบัญชี UPDATE X = X + 2,000 ทำให้ยอดเงินในขณะนี้มีค่าเท่ากับ 12,000 ธุรกรรม T1 ทำงานต่อเนื่องไปจนมีการปรับปรุงข้อมูลลงในจานบันทึกข้อมูลสำเร็จ อย่างไรก็ตาม ระบบจัดการฐานข้อมูลยังไม่ได้ทำการยอมรับหรือ COMMIT ธุรกรรม ซึ่งในขณะที่จะทำการ COMMIT ธุรกรรม T1 นั้น ระบบจัดการฐานข้อมูลทำการ interleave เพื่อประมวลผลธุรกรรม T2 ที่เข้ามาพร้อมๆ กัน ณ เวลา t2 เพื่ออ่านข้อมูลยอดเงิน READ (X) ได้ค่ายอดเงิน 11,000 บาทซึ่งเป็นยอดเงินจากฐานข้อมูล ถึงขณะนี้ดูเหมือนว่าธุรกรรมทั้ง 2 จะประมวลผลได้ถูกต้อง อย่างไรก็ตาม ณ เวลา t3 ระบบจัดการฐานข้อมูลเกิดการสลับการดำเนินการประมวลผลธุรกรรม interleave กลับไปยังธุรกรรม T1 แต่เกิดข้อผิดพลาดที่ธุรกรรม T1 ขึ้น ถึงแม้จะเกิดการเขียนข้อมูลลงไปในจานบันทึกข้อมูลแล้วแต่ยังไม่ได้มีการ COMMIT สำหรับข้อผิดพลาดที่เกิดขึ้นนั้นเป็นอะไรไม่สำคัญ อาจจะเป็นการผิดพลาดที่ตัวโปรแกรมเองหรือการจัดการหน่วยความจำใดๆ ทำให้ระบบจัดการฐานข้อมูล ROLL BACK คือการเลิกทำ และกลับไปคืน

สภาพข้อมูลยอดบัญชีก่อนเริ่มธุรกรรม T1 คือยอดเงิน 10,000 บาท ต่อมาระบบจัดการฐานข้อมูลประมวลผลธุรกรรม T2 ที่เหลืออยู่ โดยใช้ข้อมูลที่ดำเนินการค้างไว้ได้แก่ยอดเงิน 12,000 บาท ซึ่งยอดเงินนี้เองเป็นยอดเงินที่อ้างอิงมาจากธุรกรรม T1 ก่อนหน้าการ ROLL BACK เมื่อคำนวณเสร็จสิ้นและปรับปรุงลงในฐานข้อมูลได้ยอดเงินสุดท้ายเป็น 11,000 บาท โดยแสดงดังตารางการดำเนินธุรกรรมต่อไปนี้

ตารางที่ 9-2 ตารางกรณีตัวอย่างการดำเนินธุรกรรมที่ก่อให้เกิดปัญหาการอ้างอิงข้อมูลที่ไม่ได้ COMMIT

เวลา	ธุรกรรม T1	ธุรกรรม T2	ยอดเงิน (X)
t1	BEGIN TRANSACTION		10000
	READ (X) (10000)		10000
	UPDATE X = X + 1000 (11000)		10000
⇨	WRITE (X) (12000)		**12000**
t2		BEGIN TRANSACTION	12000
	⇨	READ (X) (12000)	12000
t3 ⇨	Transaction failed.		**10000**
	ROLL BACK (เกิดข้อผิดพลาดขึ้น กลับไปใช้ค่าก่อนหน้าธุรกรรมคือ 10000)		
t4	⇨	UPDATE X = X – 1000 (11000)	11000
		WRITE (X) (11000)	**11000**
		COMMIT	11000

เนื่องจากธุรกรรมการฝากเงินของนายสมชายไม่สำเร็จ จึงเสมือนมีการดำเนินการธุรกรรมเพียงธุรกรรมเดียวคือธุรกรรมถอนเงินของสมศรี หากลองพิจารณาการถอนเงิน 1,000 บาทจากบัญชีที่มียอดเงิน 10,000 บาท ยอดเงินที่เหลือจากการถอนควรจะเป็น 9,000 บาท แต่กรณีตัวอย่างยอดเงินคงเหลือคือ 11,000 เกินมา 2,000 บาท ซึ่งเกิดจากการอ้างอิงถึงยอดเงินที่มีการฝากแต่ไม่ได้ COMMIT ด้วย ตัวอย่างนี้แสดงให้เห็นถึงความผิดพลาดของข้อมูลที่เกิดจากการอ้างอิงถึงข้อมูลที่ไม่ได้ COMMIT และมีการ ROLL BACK สำหรับกรณีที่ยกตัวอย่างนี้เกิดความผิดพลาดข้อมูลในแบบข้อที่ 1 ที่ได้กล่าวมาแล้วด้วย คือการเขียนทับข้อมูลที่มีการแก้ไข (ข้อมูล ROLL BACK) ความผิดพลาดในลักษณะนี้สามารถเกิดขึ้นได้โดยไม่เกิดร่วมกับการผิดพลาดตามลักษณะความผิดพลาดในข้อ 1 ได้ในกรณีที่ T2 มีการดำเนินการเขียนข้อมูลที่ได้จากการคำนวณค่า X ไว้ที่อื่นโดยไม่แก้ไขข้อมูล X

3) ปัญหาการวิเคราะห์ข้อมูลผิดพลาด

บัญชีเงินฝากที่ได้ใช้อธิบายมาก่อนหน้าเป็นบัญชีเงินฝากออมทรัพย์ กำหนดให้นายสมชายและนางสมศรีมีบัญชีเงินฝากประจำร่วมกันอีกบัญชีหนึ่ง สมมติว่ามีธุรกรรม T3 คือนายสมชายโอนเงินจำนวน 1,000 บาทจากบัญชีออมทรัพย์ที่มีเงินอยู่ 10,000 บาท เข้าไปยังบัญชีเงินฝากประจำที่มีเงินอยู่ 50,000 บาท ธุรกรรม T3 นี้จะดำเนินการสำเร็จก็ต่อเมื่อมีการหักบัญชีจากบัญชีออมทรัพย์ และเพิ่มยอดในบัญชีฝากประจำโดยสมบูรณ์ถือเป็น 1 ธุรกรรม โดยธุรกรรม T3 เกิดขึ้นพร้อมๆ กับธุรกรรม T4 ที่นางสมศรีขอดูยอดเงินทั้งหมดที่

มีในธนาคาร ซึ่งรวมถึงในบัญชีเงินฝากออมทรัพย์และบัญชีเงินฝากประจำ กำหนดให้เกิดกรณีตัวอย่างของการดำเนินธุรกรรมดังนี้

ตารางที่ 9-3 ตารางกรณีตัวอย่างการดำเนินธุรกรรมที่ก่อให้เกิดปัญหาการวิเคราะห์ข้อมูลผิดพลาด

เวลา	ธุรกรรม T3	ธุรกรรม T4	ยอดเงิน (X)	ยอดเงิน (Y)	รวม (SUM)
t1	BEGIN TRANSACTION		10000	50000	60000
	READ (X) (10000)		10000	50000	
	UPDATE X = X - 1000 (9000)		10000	50000	
	⇨ WRITE (X) (9000)		**9000**	50000	
t2		BEGIN TRANSACTION	9000	50000	
		⇨ READ (X) (9000)	9000	50000	
		READ (Y) (50000)	9000	50000	
		SUM = X + Y (14000)	9000	50000	59000
		เกิดความผิดพลาดในการวิเคราะห์ข้อมูล			
t3	READ (Y) (50000)		9000	50000	
	UPDATE Y = Y + 1000 (51000)		9000	50000	
	⇨ WRITE (Y) (51000)		9000	**51000**	
	COMMIT		9000	51000	60000

ข้อผิดพลาดที่เกิดขึ้นเกิดจากการที่ธุรกรรมการแสดงยอดทรัพย์สินรวมอ่านยอดคงเหลือระหว่างที่ยอดเงินในบัญชีออมทรัพย์ถูกหักออก แต่ยังไม่ได้เพิ่มในบัญชีเงินฝากประจำ

เมื่อยอดเงินในบัญชีเงินฝากออมทรัพย์แทนด้วย X และยอดเงินในบัญชีเงินฝากประจำแทนด้วย Y ธุรกรรมเริ่มจากการที่ธุรกรรม T3 อ่านยอดบัญชีเงินฝากออมทรัพย์ READ (X) ได้ค่า 10,000 จากนั้นทำการหักยอดออก 1,000 บาท UPDATE X = X – 1,000 เพื่อที่จะโอนไปยังบัญชีเงินฝากประจำ ยอดเงินฝากที่เหลือในบัญชีเงินฝากประจำได้ถูกปรับปรุงในฐานข้อมูล WRITE (X) ซึ่งเป็นจำนวนเงิน 9,000 บาทที่เหลืออยู่ในตอนนี้เองที่ธุรกรรม T4 ที่ดำเนินการพร้อมๆ กัน โดยนางสมศรีขอดูยอดสินทรัพย์รวมจากบัญชีทั้ง 2 โดยอ่านยอดเงินคงเหลือในบัญชีเงินฝากออมทรัพย์ READ (X) ได้ 9,000 และเงินฝากประจำ READ (Y) ได้ 50,000 บาท เหตุการณ์นี้เองที่ทำให้เกิดความผิดพลาดขึ้นเนื่องจากธุรกรรมการโอนเงินยังไม่เสร็จสิ้น <u>เงินที่ต้องเพิ่มในยอดของบัญชีเงินฝากประจำจำนวน 1,000 บาทนั้นยังไม่ได้ดำเนินการ</u>เนื่องจากเกิด interleave ระบบคำนวณหายอดสินทรัพย์รวม SUM ได้ 59,000

ซึ่งในความเป็นจริงแล้วสินทรัพย์ในบัญชีเงินฝากออมทรัพย์ 10,000 บาท และเงินฝากประจำ 50,000 บาท ควรจะรวมกันได้ 60,000 บาทอยู่ตลอดเวลา ดังนั้นผลรวม SUM ในธุรกรรม T4 ที่ได้ 59,000 จึงเป็นการผิดพลาดที่เกิดจากภาวะพร้อมกันในลักษณะความผิดพลาดในการวิเคราะห์ข้อมูล

การแก้ไขปัญหาที่เกิดจากภาวะพร้อมกันให้คำนึงถึงผลลัพธ์ที่มีลักษณะ serializibility ดังนี้

9.2.3 Serializibility

Serializabitity คือการจัดการภาวะพร้อมกันให้มีผลลัพธ์เหมือนกับการที่ธุรกรรมแต่ละธุรกรรมทำงานต่อกัน โดยธุรกรรมหนึ่งดำเนินการเสร็จสิ้นก่อน แล้วต่อด้วยธุรกรรมอีกธุรกรรมหนึ่ง ถ้าการจัดการธุรกรรมสามารถทำให้ผลลัพธ์ออกมาได้ดังนี้ทำให้มั่นใจได้ว่าข้อมูลจะมีความถูกต้อง เนื่องจากการที่ธุรกรรมหนึ่งดำเนินการเสร็จสิ้นแล้วต่อด้วยการดำเนินการอีกธุรกรรมหนึ่งก็จะไม่ต่างกับการที่ระบบจัดการฐานข้อมูลอนุญาตให้ผู้ใช้งานใช้งานได้ทีละคนจึงจะไม่เกิดปัญหาความผิดพลาดที่เกิดจากภาวะการเกิดขึ้นพร้อมกันอย่างแน่นอน ซึ่ง serial ก็มาจากคำว่าต่อกัน/อนุกรมนั่นเอง

การจัดการธุรกรรมที่เกิดขึ้นพร้อมกันแต่ยังคงมีลักษณะของ serializibility นั้นทำให้เกิดพิธีการหรือขั้นตอนในการจัดการกับภาวะการเกิดขึ้นพร้อมกัน (concurrency control protocol) ขึ้น เพื่อจัดลำดับการทำงานของธุรกรรม โดยอาศัยหลักดังนี้

1) ถ้าธุรกรรม 2 ธุรกรรมเป็นการอ่านข้อมูล ธุรกรรมทั้ง 2 จะไม่มีผลกระทบใดๆ กับฐานข้อมูล จึงไม่ต้องมีการจัดลำดับธุรกรรม แม้ว่าธุรกรรมทั้ง 2 จะใช้ข้อมูลร่วมกัน (read only)
2) ถ้าธุรกรรม 2 ธุรกรรมเป็นการอ่านหรือเขียนข้อมูล แต่ธุรกรรมทั้ง 2 ไม่ได้ใช้ข้อมูลใดๆ ร่วมกันเลย ไม่ว่าจะอ่านหรือเขียน ธุรกรรมทั้ง 2 จึงไม่มีผลกระทบต่อกัน ไม่ต้องมีการจัดลำดับธุรกรรม (read-write on different items)
3) ถ้าธุรกรรมหนึ่งมีการดำเนินการเขียนข้อมูลแล้ว ไม่ว่าอีกธุรกรรมจะทำการอ่านหรือเขียนข้อมูลบนข้อมูลเดียวกัน อาจก่อนให้เกิดปัญหาข้อมูลผิดพลาดหรือขัดแย้งกัน ภาวะพร้อมกันของธุรกรรมในลักษณะนี้มีความจำเป็นที่เราจะต้องจัดลำดับธุรกรรมก่อน-หลัง (read-write on the same item)

การจัดการกับปัญหาภาวะพร้อมกันสามารถอธิบายได้ตามหัวข้อต่อไป

9.2.4 การจัดการปัญหาภาวะพร้อมกันด้วยเทคนิคการปิดกั้น

การจัดการกับปัญหาภาวะพร้อมกันมีหลายวิธี วิธีที่นิยมใช้กันมากคือเทคนิคที่เรียกว่าเทคนิคการปิดกั้น (lock technique) หลักการทำงานของเทคนิคการปิดกั้นนี้อาศัยการป้องกันข้อมูลที่มีการใช้งานร่วมกัน โดยไม่อนุญาตให้ธุรกรรมหรือผู้ใช้คนอื่นเข้าถึงข้อมูลได้ในระหว่างที่มีการเปลี่ยนแปลงข้อมูลในธุรกรรมหนึ่ง โดยธุรกรรมอื่นจะเข้าถึงข้อมูลได้ก็ต่อเมื่อธุรกรรมที่ดำเนินการกับฐานข้อมูลอยู่แล้วเสร็จ โดยจะเป็นการกำหนดลำดับการทำงาน ก่อน-หลังของขั้นตอนย่อยๆ ของธุรกรรม หัวข้อต่อไปนี้อธิบายถึงประเภทของการปิดกั้น ระดับของการปิดกั้นข้อมูล แล้วอธิบายถึงวิธีการทำงานของเทคนิคการปิดกั้นนี้

9.2.4.1 ประเภทของการปิดกั้น

การปิดกั้นข้อมูลด้วยเทคนิคการปิดกั้นแบ่งออกเป็น 2 ประเภท ได้แก่

1) การปิดกั้นแบบร่วม (Shared Lock หรือ S Lock หรือ Read Lock)

เป็นการปิดกั้นข้อมูลที่ยินยอมให้ธุรกรรมต่างๆ สามารถเข้าถึงและใช้งานข้อมูลร่วมกันโดยพร้อมๆ กัน โดยอนุญาตให้ธุรกรรมต่างๆ อ่านข้อมูลเท่านั้น ไม่อนุญาตให้ธุรกรรมใดธุรกรรมหนึ่งเปลี่ยนแปลงข้อมูลที่ถูกปิดกั้นนั้น

2) การปิดกั้นแบบเฉพาะ (Exclusive Lock หรือ X Lock หรือ Write Lock)

เป็นการปิดกั้นข้อมูลที่มีเพียงธุรกรรมเดียวเท่านั้นสามารถเข้าใช้ข้อมูลได้ ธุรกรรมอื่นๆ จะไม่สามารถเข้าถึงข้อมูลได้เลย แม้แต่จะทำการอ่านข้อมูลที่ถูกปิดกั้นแบบเฉพาะนั้นๆ ก็ไม่สามารถทำได้ โดยการปิดกั้นแบบ X Lock นี้จะใช้ในธุรกรรมที่มีการแก้ไขข้อมูล

9.2.4.2 ระดับของการปิดกั้น (Levels of Locking หรือ Granularity of Data Items)

ระดับของการปิดกั้นหมายถึงขอบเขตของการปิดกั้นในมุมมองของระดับชั้นของข้อมูล ซึ่งโดยทั่วไประดับของการปิดกั้นในระบบจัดการฐานข้อมูลแบ่งเป็นระดับได้ดังนี้

1) การปิดกั้นฐานข้อมูล (Database Lock)

เป็นการปิดกั้นข้อมูลทั้งฐานข้อมูล ซึ่งจะไม่อนุญาตให้ธุรกรรมอื่นๆ แก้ไขข้อมูลหรืออ่านข้อมูลใดๆ ในฐานข้อมูลเลยตามแต่ประเภทของการปิดกั้น การปิดกั้นในลักษณะนี้มักไม่เกิดขึ้นบ่อย อาจจะต้องใช้ในกรณีการสำรองข้อมูลที่สถานะข้อมูลทั้งฐานข้อมูลต้องสอดคล้องต้องกัน

2) การปิดกั้นตาราง (Table Lock)

เป็นการปิดกั้นข้อมูลทั้งตารางข้อมูล ส่วนใหญ่แล้วจะกระทำในกรณีที่มีการแก้ไขโครงสร้างของตารางเช่น เพิ่ม แก้ไข หรือลดคอลัมน์

3) การปิดกั้น Page หรือ Block หรือ Database Space (Page/Block/Database Space Lock)

ระบบจัดการฐานข้อมูลจัดการข้อมูลในลักษณะกลุ่มของข้อมูลด้วย Page หรือ Block หรือ Database Space ที่มีประสิทธิภาพมากกว่าการจัดการทีละแถวย่อยๆ ในตารางข้อมูลหนึ่งตารางอาจประกอบด้วยกลุ่มข้อมูลหลายๆ Page การปิดกั้น Page นี้ ระบบจัดการฐานข้อมูลจะปิดกั้นข้อมูลทั้ง Page ซึ่งประกอบด้วยข้อมูลหลายแถว การปิดกั้นระดับ Page นี้เป็นการดำเนินการที่กระทำบ่อยๆ ของระบบจัดการฐานข้อมูลแทนการปิดกั้นในระดับแถว โดยปิดกั้นทั้ง Page ที่มีแถวข้อมูลที่จะทำการปิดกั้นเนื่องจากจะทำได้รวดเร็วกว่า

4) การปิดกั้นแถว (Row Lock หรือ Record Lock)

เป็นการปิดกั้นเฉพาะแถวข้อมูลหรือระเบียนข้อมูลที่กำหนด ระเบียนอื่นๆ จะไม่ได้รับผลกระทบจากการปิดกั้น

5) การปิดกั้นคอลัมน์ (Column Lock)

เป็นการปิดกั้นเฉพาะคอลัมน์ คอลัมน์อื่นๆ จะไม่ถูกปิดกั้น การใช้งานการปิดกั้นในลักษณะนี้จะใช้ในการปิดกั้นคอลัมน์ที่มีการแก้ไขข้อมูลบ่อยครั้ง เช่น คอลัมน์จำนวนสินค้าคงคลังในตารางสินค้า ซึ่งจะถูกแก้ไขอยู่ตลอดเวลาที่มีธุรกรรมการซื้อขาย แต่คอลัมน์ชื่อสินค้า คำอธิบายสินค้าและอื่นๆไม่ได้รับการแก้ไขบ่อย

การปิดกั้นข้อมูลด้วยรูปแบบต่างๆ ในระบบจัดการฐานข้อมูลสามารถดำเนินการได้ด้วยคำสั่ง SQL ซึ่งมีรูปแบบดังนี้

```
LOCK TABLE <table_name> IN <SHARE | EXCLUSIVE> MODE;
```

การจะใช้การปิดกั้นรูปแบบใดนั้นต้องคำนึงถึงวัตถุประสงค์และผลกระทบที่ผู้พัฒนาระบบฐานข้อมูลและผู้ดูแลฐานข้อมูล (DBA) ต้องพิจารณา

ตัวอย่างที่ 9.1 LOCK

การปิดกั้นตารางนักศึกษา โดยใช้การปิดกั้นประเภท S Lock ซึ่งอนุญาตให้ธุรกรรมต่างๆ อ่านข้อมูลจากตารางนักศึกษาได้แต่ไม่สามารถแก้ไขได้ แสดงด้วย SQL ดังนี้

LOCK TABLE students IN SHARE MODE;

9.2.4.3 วิธีการดำเนินการปิดกั้น

วิธีดำเนินการปิดกั้นข้อมูลอาศัยหลักการปิดกั้นแบบ 2 ช่วง (2 Phase Locking—2PL) คือ

1) **ช่วงขยาย (Growing Phase)**
   เป็นการกำหนดการปิดกั้นข้อมูลที่จะต้องทำการเข้าถึง โดยข้อมูลที่จะได้รับการแก้ไขให้ปิดกั้นแบบ X-LOCK ส่วนข้อมูลที่อ่านเพียงอย่างเดียวไม่ได้รับการแก้ไขในธุรกรรมให้ใช้ S-LOCK ระยะนี้จะไม่มีการปลดการปิดกั้นใดๆ
2) **ช่วงถดถอย (Shrinking Phase)**
   หลังจากการดำเนินคำสั่งย่อยๆ ของธุรกรรมเสร็จสิ้นเรียบร้อยแล้ว ช่วงนี้เป็นช่วงของการปลดการปิดกั้นข้อมูลที่ได้ปิดกั้น ในช่วงนี้จะไม่มีการปิดกั้นข้อมูลเพิ่มอีก

อาศัยตัวอย่างธุรกรรม T3 และ T4 จากหัวข้อ 9.2.2 ที่นายสมชายทำการโอนเงินจำนวน 1,000 บาท ระหว่างบัญชีเงินฝากออมทรัพย์และบัญชีเงินฝากประจำ และธุรกรรมนางสมศรีแสดงยอดทรัพย์สินรวมของทั้ง 2 บัญชี ธุรกรรม T5 และT6 เป็นธุรกรรมที่ปรับเปลี่ยนจากธุรกรรม T3 และ T4 โดยการใช้เทคนิคการปิดกั้นป้องกันปัญหาที่อาจจะเกิดขึ้นจากภาวะพร้อมกันของทั้ง 2 ธุรกรรม ดังนี้

ธุรกรรมที่ 5 (T5) นายสมชายโอนเงินจากบัญชีฝากออมทรัพย์ (X) ไปยังบัญชีฝากประจำ (Y)

```
T5:     X-LOCK(X)
        READ(X)
        UPDATE X = X - 1000
        WRITE(X)
        X-LOCK(Y)
        READ(Y)
        UPDATE Y = Y + 1000
        WRITE(Y)
        UNLOCK(X)
        UNLOCK(Y)
```

ธุรกรรมที่ 6 (T6) นางสมศรีแสดงยอดสินทรัพย์รวม 2 บัญชี

```
T6:     S-LOCK(X)
        READ(X)
        S-LOCK(Y)
        READ(Y)
        SUM = X + Y
        UNLOCK(X)
        UNLOCK(Y)
```

ต่อไปนี้เป็นกรณีตัวอย่างการดำเนินธุรกรรม T5 และ T6 ที่เกิดขึ้นในเวลาใกล้เคียงกัน และเกิดการ interleave ในการประมวลผลของระบบจัดการฐานข้อมูลระหว่างธุรกรรมทั้ง 2

ตารางที่ 9-4 ตารางกรณีตัวอย่างการดำเนินธุรกรรมที่ใช้เทคนิคการปิดกั้น

เวลา	ธุรกรรม T5	ธุรกรรม T6	ยอดเงิน (X)	ยอดเงิน (Y)	รวม (SUM)
t1	BEGIN TRANSACTION		10000	50000	60000
t2	X-LOCK (X)		10000	50000	
t3	READ (X) (10000)		10000	50000	
t4	UPDATE X = X – 1000 (9000)		10000	50000	
t5	WRITE (X) (9000)		9000	50000	59000
t6		BEGIN TRANSACTION	9000	50000	
t7		S-LOCK (X)	9000	50000	
t8	LOCK (Y)	WAIT	9000	50000	
t9	READ (Y) (50000)	WAIT	9000	50000	
t10	UPDATE Y = Y + 1000 (51000)	WAIT	9000	50000	
t11	WRITE (Y) (51000)	WAIT	9000	51000	
t12	COMMIT	WAIT	9000	51000	
t13		READ (X) (9000)	9000	51000	
t14		S-LOCK (Y)	9000	51000	
t15		READ (Y) (10000)	9000	51000	
t16		SUM = X + Y	9000	51000	60000
t17		COMMIT	9000	51000	

ณ เวลา t2 มีการ LOCK ข้อมูล X แบบ X-LOCK ไว้ด้วยธุรกรรม T5 ซึ่งข้อมูลอาจมีการเปลี่ยนแปลงของข้อมูล จึงไม่อนุญาตให้ธุรกรรมอื่นเข้าถึงข้อมูล X หรือยอดบัญชีเงินฝากออมทรัพย์ของนายสมชายและนางสมศรีได้ ที่เวลา t5 ข้อมูลของยอดบัญชีเงินฝากออมทรัพย์เหลือเพียง 9,000 บาท ในขณะนี้เองหาธุรกรรม T6 อ่านข้อมูล X และ Y ในกรณีไม่มีการปิดกั้นข้อมูล ผลรวมของยอดเงินในบัญชีจะผิดพลาด แต่ในตัวอย่างนี้มีการใช้เทคนิคการปิดกั้น พิจารณาที่เวลา t6 มีเกิดการ interleave ของระบบจัดการฐานข้อมูลไปประมวลผลธุรกรรม T6

ณ <u>เวลา t7 เป็นตัวอย่างสำคัญที่แสดงว่าธุรกรรม T6 ไม่สามารถอ่านข้อมูลยอดเงินในบัญชีเงินฝากได้</u> โดยไม่สามารถปิดกั้นข้อมูล X เพื่ออ่านได้ เนื่องจากธุรกรรม T5 ปิดกั้นข้อมูล X ไว้ตั้งแต่เวลา t2 ธุรกรรม T6 จึงไม่มีทางเลือก และต้องรอจนกว่าธุรกรรม T5 จะปลดการปิดกั้นข้อมูล X เสียก่อน ธุรกรรม T6 จึงจะสามารถดำเนินต่อไปได้ ธุรกรรม T6 สามารถดำเนินการต่อได้ที่เวลา t13 หลังจากที่ธุรกรรม T5 ปลดการปิดกั้นข้อมูล

จากตัวอย่างการใช้เทคนิคการปิดกั้นดังกล่าวด้วยหลักการ 2PL นั้นเป็นการแก้ปัญหาความผิดพลาดของข้อมูลที่จะเกิดขึ้นจากภาวะพร้อมกันของธุรกรรมได้ อย่างไรก็ตามเทคนิคการปิดกันแบบ 2PL นี้ก่อให้เกิดผลกระทบข้างเคียงที่ไม่พึงประสงค์ ดังนี้

9.2.4.4 การปิดตาย (Dead Lock)

การปิดตายเป็นสภาวะที่ธุรกรรมต่างรอกันเพื่อใช้ข้อมูลที่อีกธุรกรรมหนึ่งปิดกั้นไว้ ซึ่งมีลักษณะของการดำเนินธุรกรรมดังนี้

ตารางที่ 9-5 ตารางกรณีตัวอย่างการดำเนินธุรกรรมที่เกิด deadlock

เวลา	ธุรกรรม Ta	ธุรกรรม Tb
t1	ปิดกั้น (X)	
t2		ปิดกั้น (Y)
t3	ปิดกั้น (Y) เพื่อใช้งานข้อมูล แต่ไม่สามารถปิดกั้นได้	
t4		ปิดกั้น (X) เพื่อใช้งานข้อมูล ไม่แต่สามารถปิดกั้นได้
t5	รอ (Y)	รอ (X)
t6	รอ (Y)	รอ (X)
t7	รอ (Y)	รอ (X)

จากลักษณะการดำเนินธุรกรรมดังกล่าว ทั้งธุรกรรม Ta และ Tb ต่างก็ใช้ข้อมูลหรือระเบียน X และ Y เช่นกัน แต่ปรากฏว่า Ta สามารถปิดกั้นข้อมูล X ได้ก่อน แต่ยังไม่ทันได้ปิดกั้นข้อมูล Y ในขณะที่ธุรกรรม Tb สามารถปิดกั้นข้อมูล Y ได้ก่อน ธุรกรรมทั้ง 2 พยายามปิดกั้นข้อมูลอีกข้อมูลหนึ่งจนต้องรอไปเรื่อยๆ การรอนี้จะไม่มีทางสิ้นสุดได้ และธุรกรรมทั้ง 2 จะไม่สำเร็จ ปัญหาที่เกิดขึ้นนี้เรียกว่าปัญหาการปิดตายของธุรกรรม (transaction dead lock)

พิจารณากรณีตัวอย่างการเกิดการปิดตายของธุรกรรม T5 และ T6 ของนายสมชายและนางสมศรี โดยมีการปรับเปลี่ยนลำดับคำสั่งย่อยของการดำเนินธุรกรรมเล็กน้อยเพื่อความสะดวกในการยกตัวอย่าง โดยธุรกรรม T6 ทำการปิดกั้นข้อมูล Y ก่อน X ดังนี้

ตารางที่ 9-6 ตารางกรณีตัวอย่างการดำเนินธุรกรรมที่เกิด deadlock

เวลา	ธุรกรรม T5	ธุรกรรม T6
t1	BEGIN TRANSACTION	
t2	X-LOCK (X)	
t3	READ (X)	
t4	UPDATE X = X – 1000	
t5	WRITE (X)	
t6		BEGIN TRANSACTION
t7		S-LOCK (Y)
		READ (Y)
t8		S-LOCK (X)
t9	LOCK (Y)	WAIT
t10	WAIT	WAIT
t11	WAIT	WAIT
t12	WAIT	WAIT

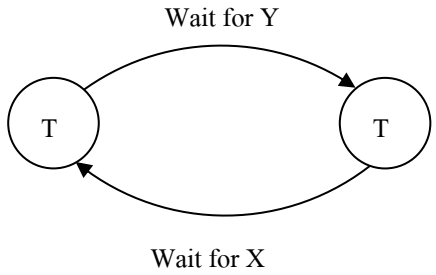
วิธีที่จะจัดการกับปัญหาการปิดตายนั้นทำได้ 2 วิธีใหญ่ๆ คือการป้องกันไว้ล่วงหน้า กับการแก้ปัญหาในกรณีที่เกิดขึ้นแล้ว

1) การป้องกันการปิดตาย

วิธีการป้องกันการปิดตายทำได้โดยการ ปิดกั้นข้อมูลล่วงหน้าทุกข้อมูลที่จะใช้ให้หมดเสียก่อนที่จะดำเนินการใดๆ กับข้อมูลไม่ว่าจะเป็นการอ่าน แก้ไข หรือเขียน และจะต้องให้ธุรกรรมนั้นๆ ปิดกั้นข้อมูลให้เสร็จเสียก่อน วิธีการนี้ทำได้ค่อนข้างยากเนื่องจากจะต้องทราบข้อมูลทุกข้อมูลที่จะใช้งานทั้งธุรกรรม

2) การแก้ปัญหาหลังจากเกิดการปิดตาย

วิธีการแก้ปัญหาสามารถทำได้หลายวิธี เช่นการใช้ Wait-for-Graph (WFG) ซึ่งเป็นการเก็บข้อมูลว่าธุรกรรมใดกำลังปิดกั้นข้อมูลตัวใดอยู่ จากนั้นอาศัยการสร้างรูปกราฟความสัมพันธ์ของธุรกรรม เมื่อใดก็ตามที่รูปกราฟเป็นรูปปิดวงกลม เช่น ธุรกรรม Ta รอข้อมูล Y จากธุรกรรม Tb และธุรกรรม Tb รอข้อมูล X จากธุรกรรม Ta ก็จะเกิดเป็นกราฟวงกลม (cyclic graph) วนกลับมาที่เดิม ทำให้สามารถตัดสินใจให้ธุรกรรมใดดำเนินต่อไปก่อนได้



ภาพที่ 9-1 Wait-for-Graph

9.2.5 เทคนิคอื่นๆ ที่ใช้ในการแก้ปัญหาภาวะพร้อมกัน

ยังมีเทคนิคอื่นๆ อีกที่สามารถแก้ปัญหาภาวะพร้อมกันได้นอกจากเทคนิคการปิดกั้น เช่น เทคนิคการทำเวอร์ชัน เทคนิค optimistic หรือเทคนิคผสมผสานกัน เป็นต้น เทคนิคอื่นๆ ที่นิยมใช้กันนั้นสามารถอธิบายได้โดยง่าย กล่าวคือ การดำเนินคำสั่งย่อยๆ ของธุรกรรมต่างๆ จะไม่มีการปิดกั้นข้อมูล แต่จะปล่อยให้ธุรกรรมดำเนินไปพร้อมๆ กัน และอาศัยกลไกสำคัญคือการจดบันทึกเวลาเริ่มต้น และเวลาที่ข้อมูลแต่ละข้อมูลถูกปรับปรุง ในกรณีที่ข้อมูลๆ หนึ่งที่กำลังจะถูกเรียกใช้ กลับถูกธุรกรรมอีกธุรกรรมหนึ่งแก้ไขข้อมูลไปแล้ว ธุรกรรมอีกธุรกรรมจะทราบว่าเกิดการแย่งข้อมูลกันขึ้น โดยดูจากเวลาที่ธุรกรรมเริ่มต้น และเวลาที่ข้อมูลที่กำลังจะถูกเรียกใช้ได้รับการแก้ไข เมื่อเกิดเหตุการณ์ดังกล่าว ธุรกรรมที่มาทีหลังจะ ROLL BACK ซึ่งการ ROLL BACK สามารถเกิดขึ้นได้ในขณะที่ธุรกรรมตรวจพบการเกิดภาวะพร้อมกันหรือก่อนทำการ COMMIT ในขั้นตอนสุดท้าย ขึ้นอยู่กับเทคนิคที่ใช้

กลไกที่รองรับการป้องกันและแก้ไขข้อผิดพลาดของข้อมูลที่เกิดจากปัญหาการเกิดภาวะพร้อมกันนี้มีการประยุกต์ใช้ในระบบฐานข้อมูลต่างๆ กันไปตามผู้ผลิตแต่ละราย นอกจากนี้การสร้างธุรกรรมต่างๆ ผู้พัฒนาระบบควรเลือกใช้เทคนิคในการควบคุมการเกิดภาวะพร้อมกันให้สอดรับกับกลไกที่ระบบจัดการฐานข้อมูลมีมาให้

9.3 การกู้ข้อมูล (Data Recovery)

ในกรณีที่ระบบจัดการฐานข้อมูลดำเนินการคำสั่งย่อยๆ ในธุรกรรมยังไม่แล้วเสร็จ มีการเปลี่ยนแปลงข้อมูลบางส่วนแล้วระบบเกิดการขัดข้อง ข้อมูลที่อยู่ในฐานข้อมูลอาจมีสภาพไม่สอดคล้องต้องกันได้ เช่นการโอนเงินจากบัญชีหนึ่งไปยังอีกบัญชีหนึ่ง หากธุรกรรมได้ทำการหักยอดบัญชีต้นทางไปแล้ว แต่ระบบยังไม่ได้ทำการเพิ่มยอดที่บัญชีปลายทาง แต่เกิดความผิดพลาดของระบบเสียก่อน เช่นไฟดับ หรือเครื่องล้มเหลว (crash) เป็นต้น ในที่นี้เราต้องสร้างกลไกในการรองรับธุรกรรมเพื่อที่จะกำหนดว่าเมื่อระบบจัดการฐานข้อมูลกลับมาทำงานได้ตามปกติแล้วจะจัดการอย่างไรกับธุรกรรมที่ยังทำค้างไว้ ซึ่งเรียกว่าการกู้ข้อมูล (data recovery)

วิธีการกู้ข้อมูลที่เกิดจากการขัดข้องของระบบจากสาเหตุต่างๆ นั้น อาศัยหลักการที่คล้ายคลึงกันแต่ยังมีรายละเอียดที่แตกต่างกันบ้าง ขึ้นอยู่กับสาเหตุของความขัดข้องนั้นๆ ในหัวข้อนี้จะได้อธิบายถึงสาเหตุของความขัดข้องประเภทต่างๆ และวิธีการที่ใช้ในการกู้ข้อมูลจากสาเหตุแต่ละประเภท

9.3.1 ประเภทของความขัดข้อง

สาเหตุที่ทำให้เกิดความขัดข้องของธุรกรรมแบ่งเป็น 3 ประเภทใหญ่ๆ ได้แก่

1) **ความขัดข้องที่เกิดจากระบบคอมพิวเตอร์ล้มเหลว (System Crash or Failure)**
   เป็นความผิดพลาดของฮาร์ดแวร์หรือซอฟต์แวร์ที่ทำให้เกิดความผิดพลาดระหว่างการดำเนินการธุรกรรมใดๆ เช่นการเขียนทับกันของข้อมูลในหน่วยความจำของเครื่อง ข้อมูลบางส่วนในหน่วยความจำหายไป หรือไฟดับและไม่มีอุปกรณ์สำรองไฟ เป็นต้น
2) **ความขัดข้องที่เกิดจากข้อผิดพลาดของธุรกรรม (Transaction Error)**
   เป็นความผิดพลาดที่เกิดจากคำสั่งในธุรกรรมเอง เช่นความผิดพลาดที่เกิดจากการไม่ได้ตรวจสอบชนิดของข้อมูลนำเข้าเสียก่อน มีการนำตัวอักษรมาคำนวณเหมือนตัวเลข หรือการหารค่าด้วย 0 เป็นต้น
3) **ความขัดข้องที่เกิดจากอุปกรณ์จัดเก็บข้อมูลหรือสื่อผิดพลาด (Storage or Media Error)**
   เป็นความผิดพลาดที่เกิดจากอุปกรณ์จัดเก็บ เช่นฮาร์ดดิสก์/จานบันทึกข้อมูล ได้รับความเสียหาย เขียนอ่านข้อมูลไม่ได้บนผิวจานบันทึกบางช่วงที่จัดเก็บฐานข้อมูล เป็นต้น

9.3.2 วิธีการกู้ข้อมูล

วิธีการกู้ข้อมูลขึ้นอยู่กับสาเหตุของการเกิดการขัดข้องแต่ละประเภท ดังนี้

1) การกู้ข้อมูลจากความขัดข้องที่เกิดจากระบบคอมพิวเตอร์ล้มเหลว (Data Recovery from System Crash or Failure)

ความล้มเหลวของระบบอาจทำให้ธุรกรรมเปลี่ยนแปลงข้อมูลในอุปกรณ์จัดเก็บไปแล้วบางส่วน แต่คำสั่งย่อยที่เหลือในธุรกรรมยังไม่แล้วเสร็จ แต่ข้อมูลที่ใช้ในธุรกรรม ตลอดจนสถานะของธุรกรรมในหน่วยความจำสูญหายไปแล้ว

เทคนิคที่เรานิยมใช้ในการจัดการกับธุรกรรมที่ขัดข้องได้แก่เทคนิคที่เรียกว่าการลงบันทึกระบบ/ลงบันทึกรายการ (system log) ซึ่งสิ่งที่บันทึกนี้เป็นค่าต่างๆ เกี่ยวกับระบบ เช่น กิจกรรม/คำสั่ง/การดำเนินการกับข้อมูล ข้อมูลในแต่ละขั้นตอนของธุรกรรม หรือความสำเร็จ/ล้มเหลวของขั้นตอนแต่ละขั้นตอน เป็นต้น

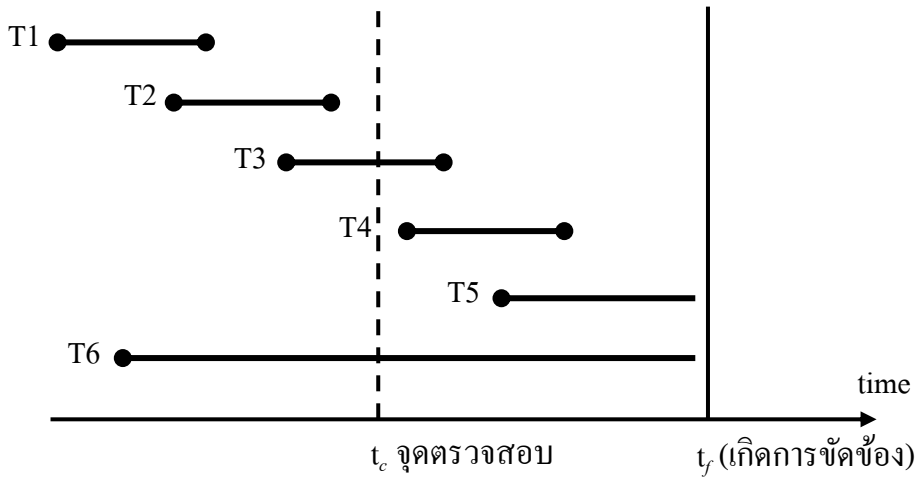
<u>การลงบันทึกระบบ (System Log)</u>

เทคนิคการลงบันทึกระบบนี้มีหลักการทำงานง่ายๆ ในแนวคิดกล่าวคือระบบจัดการฐานข้อมูลจะทำการเริ่มลงบันทึกตั้งแต่เริ่มธุรกรรม บันทึกกิจกรรมที่กระทำกับกับข้อมูล และที่สำคัญคือการบันทึกข้อมูลแต่ละข้อมูลที่ถูกแก้ไขทั้งก่อนและหลังการแก้ไขในฐานข้อมูล การใช้เทคนิค log เพื่อการกู้ข้อมูลมีหลักการง่ายๆ คือเมื่อมีการล้มเหลวของระบบ log จะมีข้อมูลของธุรกรรมที่ยังดำเนินการไม่เสร็จสิ้น ก็ให้ทำการ ROLL BACK ธุรกรรมที่ยังไม่เสร็จสิ้นทั้งหมด ส่วนการกู้จากข้อมูลสำรอง ก็นำข้อมูลสำรองขึ้นมาใช้ แล้วดำเนินธุรกรรมตาม log ที่เกิดขึ้นหลังจากการสำรองข้อมูลนั้น ในทางเทคนิคแล้ว การประยุกต์ใช้ log ยังอาศัยกลไกอีกกลไกหนึ่งที่เรียกว่าการทำจุดตรวจสอบ (checkpointing) จุดตรวจสอบนี้เป็นกลไกในการจับคู่ (synchronize) log กับสถานะของระบบจัดการฐานข้อมูลให้สัมพันธ์กัน ขณะสร้างจุดตรวจสอบ ระบบจัดการฐานข้อมูลจะบันทึกข้อมูลที่อยู่ในหน่วยความจำหลักของเครื่องไว้ด้วย เพื่อให้เป็นการแน่ใจว่า log ที่บันทึกไว้ ณ จุดตรวจสอบมีความสัมพันธ์กับสถานะจริงๆ ของฐานข้อมูล การกระทำเช่นนี้คล้ายๆ กับการสร้าง hibernate file ของคอมพิวเตอร์สมัยใหม่ ที่เมื่อทำการ hibernate แล้ว เครื่องจะเขียนข้อมูลในหน่วยความจำหลักทั้งหมดลงฮาร์ดดิสก์แล้วปิดเครื่อง เมื่อเปิดเครื่องขึ้นมาข้อมูลหน่วยความจำที่เขียนไว้จะถูกอ่านเก็บไว้ที่หน่วยความจำหลักเพื่อให้เครื่องพร้อมใช้งานได้ทันทีต่อจากสถานะเดิม เสมือนเปิดเครื่องคอมพิวเตอร์อยู่ตลอดเวลา

หลักการของการกู้ข้อมูลโดยการใช้ log มีอยู่ว่า

- ธุรกรรมใดที่ยังดำเนินการไม่สำเร็จก่อนที่จะเกิดการขัดข้องขึ้น แน่นอนว่าให้ยกเลิกธุรกรรมนั้น (UNDO/ROLL BACK)
- ธุรกรรมใดที่ดำเนินการสำเร็จ (COMMIT) ก่อนเวลาสำหรับจุดตรวจสอบ จากนั้นเกิดการขัดข้องขึ้น ธุรกรรมนั้นไม่ต้องทำใหม่ (ไม่ต้อง REDO) เนื่องจากเสมือนเราได้ยอมรับธุรกรรมนั้นแล้ว
- ธุรกรรมใดที่เริ่มดำเนินการก่อนจุดตรวจสอบ แต่สำเร็จ (COMMIT) หลังจากจุดตรวจสอบ จากนั้นเกิดการขัดข้องขึ้น ธุรกรรมประเภทนี้ต้องยกเลิกและทำใหม่ (ROLLBACK & REDO)

พิจารณาภาพจำลองการดำเนินธุรกรรมดังนี้



ภาพที่ 9-2 การดำเนินธุรกรรมพร้อมๆ กันโดยแสดงจุดตรวจสอบ t_c และเวลาที่เกิดการขัดข้อง t_f

แผนภาพดังกล่าวมีลักษณะคล้ายคลึงกับ Gantt Chart ซึ่งเป็นการแสดงว่าธุรกรรมแต่ละธุรกรรมเริ่มต้นที่เวลาใด ดำเนินธุรกรรม และสิ้นสุดที่เวลาใด โดยธุรกรรมเริ่มจากเวลาที่อยู่ทางซ้ายของแผนภาพ t_c คือเวลาที่มีการสร้างจุดตรวจสอบ t_f คือเวลาที่เกิดการขัดข้องของธุรกรรม

ธุรกรรม T1 และ ธุรกรรม T2 เกิดขึ้นและสำเร็จเรียบร้อยแล้ว ที่จุดตรวจสอบมีการบันทึกข้อมูลสถานะของระบบเก็บไว้ เมื่อเกิดการขัดข้องขึ้นที่เวลา t_f ธุรกรรมทั้ง 2 จึงไม่จำเป็นต้องทำใหม่หรือไม่จำเป็นต้องดำเนินการใดๆ กับธุรกรรมทั้ง 2

ธุรกรรม T3 นั้น แม้ว่าจะเริ่มก่อนจุดตรวจสอบ แต่จะมีสถานะเช่นเดียวกับธุรกรรม T4 ที่ COMMIT แล้ว แม้ว่าธุรกรรมทั้ง 2 สำเร็จแล้วแต่ธุรกรรมทั้ง 2 นี้ มีรายละเอียดปลีกย่อยที่จะต้องดำเนินการ กล่าวคือ ธุรกรรมต่างๆ เกิดขึ้นพร้อมๆ กัน เราไม่ทราบว่าธุรกรรม T3 และ T4 ได้แก้ไขข้อมูลอะไรในหน่วยความจำหลักไว้บ้าง ซึ่งอาจมีผลกระทบกับธุรกรรมที่กำลังดำเนินการอยู่จึงเป็นที่มาของการสร้างจุดตรวจสอบเก็บสถานะของระบบจัดการฐานข้อมูล

วิธีการจัดการกับธุรกรรมทั้ง 2 หลังจากแก้ปัญหาการขัดข้องของระบบได้แล้วก็คือ ระบบจัดการฐานข้อมูลจะทำการหาจุดอ้างอิงที่สามารถคืนสภาพได้ ซึ่งก็คือจุดตรวจสอบ t_c นั่นเอง ธุรกรรม T3 และ T4 ควรดำเนินการใหม่อีกครั้งเพื่อให้ระบบมีสถานะเหมือนกับระบบที่ทำงานอยู่ ณ จุดตรวจสอบ โดยระบบจะทำการยกเลิก ROLL BACK และทำซ้ำ REDO ธุรกรรม T3 และ T4 (สามารถเปรียบเทียบได้กับการ hibernate ของเครื่องคอมพิวเตอร์ที่ได้อธิบายไว้ข้างต้น กิจกรรมใดที่ดำเนินอยู่ควรทำให้สำเร็จเสมือนเครื่องคอมพิวเตอร์ไม่ได้ปิดหรือไม่ได้ขัดข้อง)

ธุรกรรม T5 และ T6 ยังไม่แล้วเสร็จเมื่อเกิดการขัดข้องของเครื่อง เราจึงไม่ทราบว่าธุรกรรมดังกล่าวจะสำเร็จหรือไม่หากไม่เกิดการขัดข้อง อาจเกิด ROLL BACK จากสาเหตุต่างๆ ธุรกรรมทั้ง 2 จึงจะต้องถูก ROLL BACK

2) การกู้ข้อมูลจากความขัดข้องที่เกิดจากข้อผิดพลาดของธุรกรรม (Data Recover from Transaction Error)

การกู้ข้อมูลจากความขัดข้องที่เกิดจากข้อผิดพลาดของธุรกรรมนี้ไม่ค่อยซับซ้อน กล่าวคือธุรกรรมใดก็ตามที่ยังดำเนินการไม่เสร็จสิ้น และยังไม่ได้ยอมรับ (COMMIT) แต่เกิดข้อผิดพลาดขึ้นเสียก่อน ไม่ว่าจะด้วยสาเหตุใดก็ตาม เช่น การ ROLL BACK จากปัญหาภาวะพร้อมกัน เป็นต้น แต่ความขัดข้องนี้ระบบจัดการฐานข้อมูลยังทำงานต่อไปได้ ระบบจัดการฐานข้อมูลจะทำการกู้โดยการเลิกทำหรือย้อนกลับ (UNDO or ROLL BACK) ให้ข้อมูลอยู่ในสภาพก่อนหน้าที่จะมีธุรกรรม การจะทำซ้ำธุรกรรมนั้นหรือไม่ขึ้นอยู่กับข้อกำหนดในการใช้งานระบบ

3) การกู้ข้อมูลจากความขัดข้องที่เกิดจากอุปกรณ์จัดเก็บข้อมูลหรือสื่อผิดพลาด (Data Recovery from Storage or Media Error)

ความขัดข้องประเภทนี้มักจะมีความเสียหายของข้อมูลมากกว่า 2 ประเภทแรก และมีความซับซ้อนในการกู้คืนสภาพข้อมูล กล่าวคือเราไม่สามารถที่จะดำเนินการเพียงนำ log ของธุรกรรมมาดำเนินการทำซ้ำหรือยกเลิกขั้นตอนใดๆ เท่านั้น เพราะข้อมูลบางส่วนที่อยู่ในอุปกรณ์จัดเก็บสูญหายไปแล้ว วิธีการกู้คืนสภาพข้อมูลทำได้โดยการนำข้อมูลที่ได้ทำการสำรอง (backup) ไว้ มาใช้งานจากนั้นทำการกู้ข้อมูลสำรองให้มีค่าของข้อมูลใกล้เคียงกับข้อมูลล่าสุดมากที่สุดโดยการดำเนินการตาม log ที่ได้บันทึกไว้ตั้งแต่สำรองข้อมูล จะเห็นว่า log ของระบบจัดการฐานข้อมูลมีความสำคัญมาก ไฟล์ log นี้มักจะสร้างไว้หลายชุด และจัดเก็บไว้หลายๆ ที่เพื่อกระจายความเสี่ยงต่อภัยคุกคามต่างๆ

9.4 ความมั่นคงปลอดภัยของข้อมูล

ความมั่งคงปลอดภัยของข้อมูลเป็นเรื่องที่เกี่ยวกับการสร้างกลไกในการป้องกันข้อมูลจากภัยคุกคามต่างๆ ที่ไม่เพียงแต่จะก่อให้เกิดความเสียหายต่อตัวข้อมูลเอง แต่รวมถึงเป็นความเสียหายที่เป็นผลพวงที่เกิดกับข้อมูลจากภัยคุกคามนั้นๆ วัตถุประสงค์ของการป้องกันภัยคุกคามก็เพื่อรักษาความถูกต้องหรือความมีบูรณภาพของข้อมูล (integrity) เพื่อคงไว้ซึ่งความลับของข้อมูลที่ปกปิด (confidential) และเพื่อให้ข้อมูลมีอยู่ให้ใช้งานเมื่อต้องการ (availability) ในหัวข้อนี้จึงจะได้กล่าวถึงภัยคุกคามที่สำคัญ และวิธีการแต่ละวิธีที่จะป้องกันภัยคุกคามได้อย่างมีประสิทธิภาพ

9.4.1 ภัยคุกคาม

ภัยคุกคามคือการกระทำใดๆ ที่เกิดจากความตั้งใจหรืออาจไม่ได้ตั้งใจ รวมถึงเหตุการณ์ที่เป็นภัยต่างๆ ที่เกิดขึ้นแล้วสร้างความเสียหายให้กับข้อมูล และแน่นอน ย่อมต้องมีผลเสียหายต่อเจ้าของข้อมูลและผู้ใช้ข้อมูล ต่อไปนี้จะเป็นการอธิบายภัยคุกคามรูปแบบต่างๆ ซึ่งเมื่อทราบถึงภัยคุกคามแล้วเราจะสามารถดำเนินการป้องกันภัยคุกคามที่อาจจะเกิดขึ้นได้

ภัยคุกคามมีหลากหลายรูปแบบมาก และมีรูปแบบใหม่ๆ เกิดขึ้นอยู่พอสมควร เราสามารถจัดกลุ่มภายคุกคามออกได้หลายรูปแบบ ในที่นี้จะแจกแจงภัยคุกคามออกโดยอาศัยสาเหตุของการเกิดภัยคุกคาม ได้แก่ มนุษย์ ฮาร์ดแวร์ ซอฟต์แวร์ และ ภัยพิบัติ ดังนี้

1) มนุษย์

เป็นภัยคุกคามที่เกิดจากมนุษย์ ทั้งที่ตั้งใจและไม่ได้ตั้งใจ ภัยคุกคามจากมนุษย์ที่สำคัญได้แก่

- การเข้าใช้ระบบโดยบัญชีผู้ใช้ของผู้อื่น
  หมายถึงการที่ผู้ไม่ประสงค์ดีทำการวิธีการใดๆ ในการได้มาซึ่งบัญชีในการเข้าใช้ระบบของผู้อื่นพร้อมรหัสผ่าน เช่นได้จากการแอบสังเกต การเจาะระบบ (hack) การขโมยข้อมูลที่บันทึกไว้ การได้บัญชีมาโดยบังเอิญ การใช้ซอฟต์แวร์ช่วยในการขโมยบัญชีผู้ใช้ ฯลฯ ซึ่งสามารถทำให้บุคคลที่ไม่สมควรได้รับอนุญาตเข้าถึงข้อมูลได้ ซึ่งความเสียหายที่เกิดขึ้นของภัยคุกคามประเภทนี้นั้นอาจมากมายเกินกว่าจะประเมินได้ เพราะเจ้าของบัญชีอาจจะไม่ทราบเลยก็ได้ว่าข้อมูลถูกผู้อื่นใช้งาน สูญเสียความเป็นส่วนตัว ข้อมูลอาจนำไปใช้เพื่อความได้เปรียบของคู่แข่ง หรือแม้แต่ถูกทำให้เสียหาย
- การเจาะระบบ (hack)
  การเจาะระบบคือความพยายามใดๆ ที่มีเจตนาจะเข้าถึงข้อมูลหรือเป็นผู้ใช้ของระบบให้ได้ โดยเน้นที่กิจกรรมการดำเนินการทางซอฟต์แวร์ที่จะได้มาซึ่งช่องทาง บัญชีผู้ใช้ ฯลฯ ในการเข้าถึงข้อมูล การเจาะระบบสามารถใช้วิธีการได้หลายวิธีได้แก่การอาศัยช่องว่างของระบบเพื่อหาทำเข้าควบคุมระบบ การเดาบัญชีและรหัสผู้ใช้ การใช้ซอฟต์แวร์ในการดักจับข้อมูลบัญชีผู้ใช้ การใช้ซอฟต์แวร์จำพวกโทรจัน (Trojan) ในการสร้างช่องทางการเข้าโจมตีระบบ เป็นต้น ผู้ที่เจาะระบบสำเร็จสามารถดำเนินกิจกรรมใดๆ ตามที่ต้องการในระดับที่เจาะระบบได้
- การใช้งานเกินสิทธิของตัวเองโดยบังเอิญ
  ในบางกรณีผู้ดูแลระบบกำหนดสิทธิของผู้ใช้ไว้ผิดพลาด เช่น ผู้ใช้บางคนสามารถเข้าถึงข้อมูลตารางพนักงานได้แต่ผู้ดูแลและผู้พัฒนาระบบไม่ได้กำหนดสิทธิป้องกันไม่ให้เข้าถึงข้อมูลเงินเดือนของพนักงานคนอื่น เป็นต้น
- การลักลอบดักจับข้อมูลในเครือข่ายการสื่อสาร
  เป็นการใช้ซอฟต์แวร์คอยดักจับข้อมูล หรือแม้แต่นำอุปกรณ์เชื่อมต่อเข้ากับเครือข่ายเพื่อดักจับข้อมูลต่างๆ ในเครือข่ายคอมพิวเตอร์ ข้อมูลที่ได้อาจจะเป็นข้อมูลที่สำคัญ รวมถึงอาจเป็นข้อมูลในการเข้าใช้ระบบ
- การเข้าถึงระบบทางกายภาพ
  หมายถึงการเข้าถึงตัวฮาร์ดแวร์ของระบบสารสนเทศ ฐานข้อมูล หรือระบบเครือข่ายในทางกายภาพ อาจมีการทำลายอุปกรณ์เหล่านี้ให้เสียหาย ขโมยอุปกรณ์ หรือความพยายามที่จะเจาะระบบ
- การโจมตีเพื่อให้ระบบไม่สามารถให้บริการได้

ระบบฐานข้อมูลควรที่จะสามารถให้บริการได้อยู่ตลอดเวลาที่ต้องการ แต่ผู้ไม่ประสงค์ดีอาจจะอาศัยการร้องขอข้อมูลจากระบบมากเกินความสามารถที่ระบบจะรองรับได้ ทำให้ระบบไม่สามารถให้บริการได้ในที่สุด

2) ฮาร์ดแวร์

เป็นภัยคุกคามที่เกิดจากความเสียหายที่เกิดจากตัวอุปกรณ์ทางด้านกายภาพ

- ระบบคอมพิวเตอร์ ระบบการสื่อสาร หรืออุปกรณ์จัดเก็บเสียหาย
  เป็นเรื่องปกติที่อุปกรณ์ฮาร์ดแวร์ใช้งานแล้วจะเสื่อสภาพตามอายุการใช้งาน หรือความผิดปกติของอุปกรณ์จากการผลิต การขนส่ง หรือสภาพแวดล้อมและกิจกรรมอื่นๆ ทำให้อุปกรณ์เสียหาย ทำให้ข้อมูลสูญหาย ระบบไม่สามารถใช้งานได้ หรือเกิดช่องว่างของความมั่นคงปลอดภัยของระบบ

3) ซอฟต์แวร์

ภัยคุกคามที่เกิดจากซอฟต์แวร์นั้นโดยแท้จริงแล้วก็เป็นผลจากการกระทำของมนุษย์ที่เป็นผู้พัฒนาซอฟต์แวร์เช่นเดียวกัน ภัยคุกคามที่เกิดจากซอฟต์แวร์ที่สำคัญได้แก่

- ไวรัสคอมพิวเตอร์ หรือโปรแกรมประสงค์ร้าย (computer virus, malware, worm, etc.)
  เป็นซอฟต์แวร์ที่อาศัยวิธีต่างๆ ในการแพร่กระจายไปยังระบบคอมพิวเตอร์อื่นๆ แล้วสร้างความเสียหายให้กับข้อมูล ลบและเปลี่ยนแปลงข้อมูลโดยไม่ได้รับอนุญาต ตลอดจนสร้างช่องทางในการเข้าใช้ระบบจากผู้ไม่ประสงค์ดี
- ความผิดพลาดของตัวระบบสารสนเทศ หรือ ระบบจัดการฐานข้อมูลเอง
  ได้แก่ความผิดพลาดในการดำเนินงานของระบบ เช่นการเขียนและอ่านข้อมูลจากหน่วยความจำหลักที่ไม่มีข้อมูลอยู่ หรือความผิดพลาดจากการพัฒนาโค้ดโปรแกรมทำให้การจัดการข้อมูลผิดพลาดและระบบความมั่นคงปลอดภัยมีช่องโหว่
- ความล้มเหลวของระบบรักษาความมั่นคงปลอดภัย
  ซอฟต์แวร์หลายประเภท รวมถึงหน่วยย่อยในระบบจัดการฐานข้อมูลบางหน่วยถูกออกแบบมาสำหรับรักษาความมั่นคงปลอดภัยของข้อมูล เช่น หน่วยที่ใช้ควบคุมการเข้าใช้ระบบ หรือซอฟต์แวร์ป้องกันการเจาะระบบ เกิดทำงานล้มเหลว ทำให้ผู้ไม่ประสงค์ดีเข้าโจมตีระบบและข้อมูลได้

4) ภัยพิบัติ

เป็นภัยคุกคามที่เกิดขึ้นจากอุบัติเหตุหรือการกระทำที่ไม่อาจควบคุมได้

- ไฟฟ้าดับหรือไฟกระชาก (power outage or power surge)
  ทำให้ธุรกรรมขัดจังหวะ หรืออุปกรณ์เสียหาย

- ไฟไหม้
  ซึ่งอาจจะเกิดจากการวางเพลิง ไฟฟ้าลัดวงจร ฯลฯ ทำให้อุปกรณ์ฮาร์ดแวร์และข้อมูลเสียหาย
- ภัยธรรมชาติ
  น้ำท่วม แผ่นดินไหว ฯลฯ ซึ่งภัยธรรมชาติบางประเภทยากที่จะควบคุม
- ภัยจากการก่อการร้าย
  ภัยจากการก่อการร้าย และภัยสงครามที่ไม่ได้สร้างความเสียหายให้กับข้อมูลเท่านั้น

ภัยคุกคามที่กล่าวมาทั้งหมดสามารถสร้างความเสียหายให้กับการใช้ข้อมูลได้มากน้อยต่างกัน แต่จะมีผลอย่างหนึ่งอย่างใดที่จะทำให้ไม่เกิดความมั่นคงปลอดภัยของข้อมูล ได้แก่การรั่วไหลของข้อมูล ข้อมูลไม่มีความเป็นบูรณภาพ ข้อมูลผิดจากที่ควรจะเป็น หรือข้อมูลเสียหาย และการไม่สามารถใช้ระบบฐานข้อมูลได้ ซึ่งเราสามารถป้องกันภัยคุกคามต่างๆ ได้ดังนี้

9.4.2 การป้องกันภัยคุกคาม

วิธีรักษาความมั่งคงปลอดภัยของระบบฐานข้อมูลที่นิยมปฏิบัติได้แก่

1) การให้สิทธิในการเข้าใช้ข้อมูลและยืนยันบุคคลผู้ใช้ (Authorization and Authentication)

การให้สิทธิในการเข้าใช้ข้อมูล (authorization) คือการให้สิทธิกับผู้ใช้ที่กำหนดให้มีสิทธิในการเข้าใช้ข้อมูลหรือวัตถุใดๆ ในฐานข้อมูล เช่นการสร้างบัญชีผู้ใช้ระบบ ทั้งนี้ยังต้องอาศัยกลไกการยืนยันบุคคลผู้ใช้เมื่อจะใช้สิทธิที่ได้รับขณะเข้าใช้ระบบที่เรียกว่าการยืนยันบุคคลผู้ใช้ (authentication) เช่น การล็อกอินเข้าใช้ระบบของผู้ใช้โดยตรวจสอบชื่อผู้ใช้กับรหัสผ่านนั่นเอง

2) การควบคุมการเข้าถึง (Access Controls)

นอกจากการให้สิทธิในการเข้าใช้ข้อมูลแล้ว เรายังสามารถกำหนดระดับของข้อมูล และกิจกรรมที่กระทำกับข้อมูลนั้นได้โดยการควบคุมการเข้าถึง เช่น การกำหนดให้ฝ่ายบุคคลสามารถอ่าน แก้ไข และเพิ่มข้อมูลของพนักงานได้ ในขณะที่ผู้ใช้ทั่วไปสามารถแสดงข้อมูลได้เพียงอย่างเดียวเท่านั้น เราสามารถกำหนดสิทธิในการดำเนินกิจกรรมของผู้ใช้ฐานข้อมูลบนตารางแต่ละตารางได้ โดยระบบจัดการฐานข้อมูลเชิงสัมพันธ์รองรับการกำหนดสิทธิด้วยคำสั่ง SQL GRANT (ให้สิทธิ) และ REVOKE (ยกเลิกหรือเพิกถอนสิทธิ) ซึ่งมีรูปแบบอย่างง่ายของคำสั่งดังนี้

<u>การให้สิทธิ</u>

```
GRANT <SELECT | INSERT | UPDATE | DELETE>
TO <user_name> ON <table_name>
```

โดย

GRANT คือคำสั่งที่ใช้ในการให้สิทธิ

<SELECT|INSERT…> คือกิจกรรมที่ต้องการจะให้สิทธิแก่ผู้ใช้ เราสามารถให้สิทธิหลายๆ สิทธิพร้อมๆ กันได้โดยใช้เครื่องหมายจุลภาค "," ขั้น กิจกรรมที่ต้องการจะให้สิทธิ

TO ประกอบคำสั่ง GRANT เพื่อระบุว่าให้สิทธิ "แก่" ผู้ใช้คนใด

user_name ชื่อผู้ใช้ฐานข้อมูลที่สร้างขึ้นไว้แล้ว

ON ประกอบคำสั่ง GRANT เพื่อระบุว่าให้สิทธิ "บน" ตารางใด

table_name ชื่อตารางผู้ใช้ได้รับสิทธิให้ทำกิจกรรมนั้นๆ

สามารถเขียนรูปแบบคำสั่งการให้สิทธิที่สามารถเข้าใจได้ง่ายขึ้นเทียบกับภาษาไทยได้ดังนี้

ให้สิทธิ <แสดง|แทรก|แก้ไข|ลบ> แก่ ชื่อผู้ใช้ บน ชื่อตาราง

ตัวอย่างที่ 9.2 การให้สิทธิ

ผู้ดูแลระบบสร้างบัญชีผู้ใช้สำหรับข้อมูลทะเบียนนักศึกษาได้แก่ registrar การให้สิทธิ registrar ในการ SELECT, INSERT, UPDATE และ DELETE ข้อมูลบนตารางลงทะเบียน Enrolled ได้ด้วยคำสั่งดังนี้

GRANT SELECT, INSERT, UPDATE, DELETE TO registrar ON enrolled

<u>การเพิกถอนสิทธิ</u>

```
REVOKE <SELECT | INSERT | UPDATE | DELETE>
FROM <user_name> ON <table_name>
```

โดย

REVOKE คือคำสั่งที่ใช้ในการเพิกถอนสิทธิ

<SELECT|INSERT…> คือกิจกรรมที่ต้องการจะให้สิทธิแก่ผู้ใช้ เราสามารถให้สิทธิหลายๆ สิทธิพร้อมกัน โดยใช้เครื่องหมายจุลภาค "," ขั้น กิจกรรมที่ต้องการจะให้สิทธิ

FROM ประกอบคำสั่ง REVOKE เพื่อระบุว่าเพิกถอนสิทธิ "จาก" ผู้ใช้คนใด

user_name ชื่อผู้ใช้ฐานข้อมูลที่สร้างขึ้นไว้แล้ว

ON ประกอบคำสั่ง REVOKE เพื่อระบุว่าให้สิทธิ "บน" ตารางใด

table_name ชื่อตารางผู้ใช้ได้รับสิทธิให้ทำกิจกรรมนั้นๆ

สามารถเขียนรูปแบบคำสั่งการให้สิทธิที่สามารถเข้าใจได้ง่ายขึ้นเทียบกับภาษาไทยได้ดังนี้

เพิกถอนสิทธิ <แสดง|แทรก|แก้ไข|ลบ> จาก ชื่อผู้ใช้ บน ชื่อตาราง

ตัวอย่างที่ 9.3 การเพิกถอนสิทธิ

การเพิกถอนสิทธิ registrar ในการ INSERT, UPDATE และ DELETE ข้อมูลบนตารางคณาจารย์ Faculty ใช้คำสั่ง

REVOKE INSERT, UPDATE, DELETE FROM registrar ON faculty

คำสั่ง GRANT และ REVOKE คือคำสั่งในกลุ่มที่ใช้นิยามข้อมูล แต่สามารถแยกออกมาเป็นกลุ่มเฉพาะนอกเหนือจาก DDL และ DML ได้ เรียกว่าภาษาควบคุม (control language) กิจกรรมที่สามารถให้และเพิกถอนสิทธิได้อาจมีมากกว่านี้ต่างกันไปตามระบบจัดการฐานข้อมูล เช่น INDEX คือการให้สิทธิในการสร้างดัชนีของตาราง

3) การใช้วิว Views

วิวคือการสร้างตารางข้อมูลจำลองเสมือนหน้าต่างครอบตารางข้อมูลที่มีอยู่จริง เช่นการสร้างตาราง faculty_public ซึ่งเป็นตารางข้อมูลคณาจารย์ที่ไม่มีคอลัมน์เงินเดือน เป็นการรักษาความลับของข้อมูลที่ควรปกปิด วิวได้กล่าวโดยละเอียดถึงการใช้งานในหัวข้อ 7.7 ในบทที่ 7

4) การสำรองและคืนสภาพข้อมูล Backup and Recovery

การสำรองข้อมูลคือการทำสำเนาของข้อมูลไว้เป็นประจำตามระยะเวลาที่กำหนด เช่น ทุกๆ วัน หรือทุกๆ สัปดาห์ เป็นต้น ซึ่งข้อมูลที่ทำสำเนาได้แก่ตัวฐานข้อมูล log หรือแม้แต่ตัวโปรแกรมประยุกต์ที่ใช้กับระบบฐานข้อมูล การสำรองข้อมูลควรจัดเก็บสำเนาไว้ในอุปกรณ์จัดเก็บอีกชุดหนึ่ง ทั้งนี้สามารถทำสำเนาไว้ได้หลายสำเนา และจัดเก็บไว้ในสถานที่ต่างๆ กันก็ได้ การสำรองข้อมูลนี้สามารถ

5) การเข้ารหัส Encryption

การเข้ารหัสหมายถึงการทำให้ข้อมูลอยู่ในรูปแบบที่อ่านเข้าใจไม่ได้ และไม่มีโปรแกรมใดๆ จะถอดรหัสให้ข้อมูลกลับมาอยู่ในรูปแบบของข้อมูลจริงก่อนการถอดรหัสนอกเสียจากว่าจะใช้คีย์ในการถอดรหัส เนื่องจากระบบจัดการฐานข้อมูลอาจจัดเก็บข้อมูลที่มีความสำคัญเป็นอย่างมากต่อบริษัท เช่นข้อมูลบัตรเครดิตของลูกค้า (ซึ่งปัจจุบันไม่ค่อยได้จัดเก็บไว้เต็มรูปแบบ) หรือข้อมูลความลับทางการค้าต่างๆ ฯลฯ ข้อมูลเหล่านี้จะมีความปลอดภัยระดับหนึ่งในกรณีที่มีผู้เจาะเข้ามาในระบบและทำการขโมยข้อมูล จะไม่สามารถนำข้อมูลไปใช้ได้โดยง่าย การเข้ารหัสสามารถทำได้ทั้งการเข้ารหัสบนฐานข้อมูลและการเข้ารหัสระหว่างการติดต่อสื่อสารในระบบ

6) การใช้เงื่อนไขบังคับบูรณภาพ

การใช้เงื่อนไขบังคับบูรณภาพเป็นการกำหนดมาตรการของการรักษาความมั่นคงปลอดภัยโดยตรงต่อความมีบูรณภาพของข้อมูล เงื่อนไขบูรณภาพอธิบายไว้อย่างละเอียดในบทที่ 4 หัวข้อ 4.3

7) การใช้ RAID

RAID (Redundant Array of Independent Disks) คือการใช้ฮาร์ดดิสก์ที่เป็นอิสระต่อกันหลายๆ ตัวเพื่อจัดเก็บข้อมูล ซึ่ง RAID นั้นมีหลายระดับ เช่น RAID0 ข้อมูลที่ถูกจัดเก็บและอ่านจากฮาร์ดดิสก์จะถูกแบ่งออกเป็นช่วงๆ ในปริมาณเท่าๆ กัน ข้อมูลแต่ละช่วงจะถูกจัดเก็บลงในฮาร์ดดิสก์พร้อมๆ กัน และอ่านขึ้นมาพร้อมๆ กันขนานกันไปทำให้การจัดการข้อมูลเป็นไปอย่างรวดเร็ว RAID1 เป็นการใช้ RAID ที่มีการสร้างข้อมูลเหมือนกันจำนวน 2 ชุด ซึ่งลักษณะของการใช้ RAID แบบนี้เองที่จะสามารถเพิ่มความมั่นคงปลอดภัยของข้อมูลได้ในกรณีที่ฮาร์ดดิสก์ตัวหนึ่งเกิดเสียหาย เรายังมีข้อมูลอีกชุดหนึ่งไว้ใช้งานได้ทันที อย่างไรก็ตาม RAID1 นี้จะต้องใช้ฮาร์ดดิสก์ที่ดุสิ้นเปลืองเนื่องจากจะต้องใช้ฮาร์ดแวร์เป็น 2 เท่า จึงมีการคิดค้น RAID แบบ

อื่นๆ อีกที่สามารถลดจำนวนอุปกรณ์จัดเก็บลงได้ แต่อาจแลกมาด้วยประสิทธิภาพที่ลดลงเพราะต้องสูญเสียไปกับการประมวลผล RAID ที่นิยมใช้ได้แก่ RAID0, RAID1 และ RAID5

8) การป้องกันทางกายภาพ

การป้องกันทางกายภาพเป็นการป้องกันอุปกรณ์ฮาร์ดแวร์ต่างๆ จากภัยคุกคามที่จะเข้าถึงตัวฮาร์ดแวร์ได้โดยตรง เช่น การควบคุมบุคคลเข้า-ออกจากห้องที่เป็นที่อยู่เครื่องคอมพิวเตอร์ที่ให้บริการฐานข้อมูล การใช้ประตูที่แน่นหนา การใช้ระบบแสกนลายนิ้วมือในการเข้าห้องของระบบ ตรวจสอบอุปกรณ์การเชื่อมต่อเพื่อป้องกันดักฟังสัญญาณ การป้องกันระบบคอมพิวเตอร์จากน้ำท่วม ไฟไหม้ ความร้อน เป็นต้น

นอกจากนี้ยังมีมาตรการอื่นๆ อีก เช่นการใช้โปรแกรมป้องกันภัยคุกคามโดยตรง การใช้อุปกรณ์สำรองไฟฟ้า ฯลฯ ซึ่งผู้ดูแลควรตรวจสอบมาตรการรักษาความมั่นคงปลอดภัยของข้อมูลให้ได้รับการปฏิบัติอยู่เสมอ

9.5 บทสรุป

ธุรกรรมหรือรายการเปลี่ยนแปลงเป็นชุดของการดำเนินงานย่อยในการดำเนินการกับฐานข้อมูลที่จะต้องทำให้สำเร็จทั้งชุด หากการดำเนินการไม่สำเร็จโดยไม่ได้รับการยอมรับการเปลี่ยนแปลง ธุรกรรมนั้นๆ จะถูกยกเลิกให้ข้อมูลที่ถูกดำเนินการแล้วกลับไปอยู่ในสถานะก่อนการดำเนินธุรกรรม การจัดการธุรกรรมมีความสำคัญในการรักษาความถูกต้องของข้อมูลให้มีบูรณภาพอยู่เสมอ

ผู้ใช้งานฐานข้อมูลสามารถใช้งานพร้อมๆ กันได้แต่อาจทำให้เกิดปัญหาจากการเกิดภาวะพร้อมกัน เนื่องจากมีการดำเนินการกับข้อมูลชุดเดียวกันอันอาจทำให้การแก้ไขข้อมูลขัดแย้งกัน เทคนิคการปิดกั้นเป็นเทคนิคที่นิยมใช้ในการแก้ปัญหาดังกล่าวโดยทำให้ธุรกรรมที่ดำเนินการพร้อมกันทำงานในลักษณะของการดำเนินการที่มี serializibility หรือดำเนินการต่อกันไป เสมือนธุรกรรมหนึ่งเริ่มต้นและดำเนินการสำเร็จก่อนที่อีกธุรกรรมจะเริ่มต้น และเสมือนหนึ่งกับว่าผู้ใช้งานคนหนึ่งใช้งานฐานข้อมูลอยู่เพียงลำพัง การใช้เทคนิคการปิดกั้นมีหลักการทำงานคือการปิดกั้นข้อมูลที่เกี่ยวข้องกับธุรกรรมไม่ให้ธุรกรรมอื่นสามารถเข้าถึงข้อมูลที่ปิดกั้นได้ อันอาจจะนำมาซึ่งปัญหาการปิดตายที่เมื่อธุรกรรมมากกว่า 2 ธุรกรรมต่างก็ปิดกั้นข้อมูลบางชุดที่ต้องใช้ในธุรกรรมของตนเอง แต่ข้อมูลส่วนหนึ่งถูกอีกธุรกรรมปิดกั้นไว้ และต่างรอที่จะปิดกั้นข้อมูลที่อีกธุรกรรมปิดกั้นได้สำเร็จ จนธุรกรรมไม่สามารถดำเนินการต่อไปได้

ในบางกรณีที่ธุรกรรมยังดำเนินการไม่แล้วเสร็จ ข้อมูลบางส่วนถูกดำเนินการแก้ไขไปแล้ว แต่เกิดการขัดข้องของระบบขึ้นทำให้ข้อมูลในฐานข้อมูลขัดแย้งกันเอง กลไกในการจัดการธุรกรรมยังสามารถทำให้เรากู้ข้อมูลจากการขัดข้องของระบบ และจัดการกับธุรกรรมที่ยังดำเนินการไม่แล้วเสร็จ เพื่อให้ข้อมูลในฐานข้อมูลกลับมามีความเป็นบูรณภาพ และต้องกันได้ในที่สุด ซึ่งเราอาศัยการลงบันทึกระบบคอยบันทึกกิจกรรมและสถานะของข้อมูลในการดำเนินการกับฐานข้อมูล ซึ่งบันทึกของระบบนี้สามารถนำมาใช้ในการกู้ข้อมูลได้

กลไกในการจัดการธุรกรรมนี้เป็นส่วนหนึ่งของการรักษาความมั่นคงปลอดภัยของฐานข้อมูลโดยเรามีความจำเป็นที่จะต้องป้องกันระบบฐานข้อมูลจากภัยคุกคามต่างๆ เพื่อไม่ให้ข้อมูลเสียหายหรือมีผู้ที่ไม่ได้รับอนุญาตให้ใช้งานเข้าใช้งานข้อมูลได้

9.6 แบบฝึกหัดท้ายบท

1. กำหนด Pseudo Code สำหรับ Transaction T1 และ T2 ซึ่งใช้ค่า X และ Y ร่วมกันดังต่อไปนี้

T1	**T2**
Begin Transaction	Begin Transaction
Read(Y)	Read(X)
Y = Y + 100	Read(Y)
Write(Y)	Y = X + Y
Read(X)	Write(Y)
X = X * 100	Commit
Write(X)	
Commit	

1.1 ให้ทำการปรับปรุง Pseudo Code ดังกล่าวโดยเพิ่มคำสั่ง Lock ลงให้ Transaction T1 และ/หรือ T2 ให้เหมาะสม เพื่อป้องกันความผิดพลาดของข้อมูลที่อาจจะเกิดขึ้นในกรณีที่เกิด Concurrency

1.2 Dead Lock ในธุรกรรมฐานข้อมูลคืออะไร จงยกกรณีตัวอย่างการกระทำการธุรกรรมจาก Transaction T1 และ T2 ที่กำหนดให้ และมีวิธีแก้ไขอย่างไร

2. ให้นักศึกษาใช้วัตถุประสงค์ของบทในการฝึกหัด โดยให้จับกลุ่ม 6 คน เพื่ออธิบายตามวัตถุประสงค์แต่ละข้อ โดยให้แต่ละคนเลือกข้อที่จะอธิบายเพียง 1 ข้อ ให้ผู้ฟังประเมินระดับความเข้าใจและการอธิบายของนักศึกษาด้วยเกณฑ์ “ควรปรับปรุง” “พอใช้” “ดี” และ “ดีมาก”

- สามารถอธิบายความหมายและความสำคัญของธุรกรรม/รายการเปลี่ยนแปลง (transaction) ได้
- สามารถอธิบายความหมายของภาวะพร้อมกัน (concurrency) …

บรรณานุกรมและเอกสารอ้างอิงประจำบท

ศุภกฤษฎิ์ นิวัฒนากูล. (2547). **เอกสารประกอบการบรรยายรายวิชา 204204 การออกแบบและพัฒนาฐานข้อมูล.** นครราชสีมา : มหาวิทยาลัยเทคโนโลยีสุรนารี.

Connolly, T. and Begg, C. (2004). **Database Systems: A Practical Approach to Design, Implementation and Management, 4th ed.** Boston : Addison-Wesley.

Ramakrishnan, R. and Gehrke, J. (2003). **Database Management Systems, 3rd ed**. Singapore : McGraw-Hill.

Elmarsi, R. and Navathe, S. B. (2006). **Fundamentals of Database Systems, 5th ed.** Boston : Addison-Wesley.

WikiPedia. “Transaction.” [Online]. Available : http://en.wikipedia.org/wiki/Transaction/ (1 September, 2007).